CMPUT 404

Web Applications and Architecture

Project: Distributed Social Networking

- Disclaimer
- Description
  - Scenario
  - Scenario Summary
  - Project Parts
- Collaboration
- User Stories
- Main Concepts
  - Authentication
  - Pagination
- Communication
  - Who talks to Who
  - Overview
  - Example (node-to-node API View)
  - IDs

- API Objects
    - Example Author Objects
    - Example Follow Request Object
    - Example Entry Object
    - Example Entries Object
    - Example Comment Object
    - Example Comments Object
    - Example Like Object
    - Example Likes Object
- API Endpoints
    - Authentication
    - Authors API
    - Single Author API
    - Following API
    - Followers API
    - Follow Request API
    - Entries API
        - Image Entries
    - Comments API
    - Commented API
    - Likes API
    - Liked API
    - Other Local APIs
    - Inbox API
- Requirements
    - API Requirements
    - Test Requirements
    - Documentation Requirements
    - Code Requirements
    - Tool Use Requirements
    - Infrastructure and Software Requirements
- Takeaways
    - Things that are allowed
- Teamwork Tips
- Submission Instructions
- Marking
    - Overall Marking
- Project Parts
    - Project Part 0: Group Formation

    - Project Part 1: Halfway Prototype
        - Requirements
        - Submission
        - Marking
    - Project Part 2: Centralized Prototype
        - Submission
        - Marking
    - Project Part 3: Distribution
        - Requirements
        - Submission
        - Marking
    - Project Part 4: Federation
        - Requirements
        - Submission
        - Marking
    - Project Part 5: Polish
        - Submission
        - Presentation
        - Marking
- License

Disclaimer

This spec is subject to change as we go through the semester!

Description

The web is fundamentally interconnected and peer to peer. There's no really great reason why we should all use facebook.com or google+ or MySpace or something like that. If these social networks came up with an API you could probably link between them and use the social network you wanted. Furthermore, you might gain some autonomy.

Thus, in the spirit of diaspora https://diasporafoundation.org/ (Diaspora was the predecessor to Mastodon) we want to build something like diaspora but far, far simpler.

This blogging/social network platform will allow the importing of other sources of information (GitHub) as well allow the distribution and sharing of entries and content.

An author sitting on one node can aggregate the entries of authors they follow on other nodes.

We are going to go with an inbox model where you push entries to your followers by sending them your entries. This is similar to activity pub: https://www.w3.org/TR/activitypub/ ActivityPub is the protocol that powers most distributed social media such as Mastodon. Activity Pub is great, but too complex for a class project.

We also won't be adding much in the way of encryption or security to this platform. We're keeping it simple and restful.

Choose at least 3 other groups to work with!

Scenario

I log into SocialDistribution. I see my stream which is filled with entries that have arrived in my inbox combined with public entries that my node knows about. I browse them and I click like on anything by my friend Steph who is on another node.

When I click like, my node sends a like object to Steph's inbox that references her entry.

I then comment on Steph's entry. This sends a comment object to Steph's inbox that references her entry.

Steph's node will process events at her inbox and record the comments and likes appropriately.

Then I write a public entry, a public service announcement (PSA) about how public service announcements are pretentious performative social media, and you shouldn't make them. The irony is lost on me. I make an unlisted image entry that contains an image for the PSA and reference it from my PSA entry. Nonetheless, my node records my entry and makes a URL for both entries and then proceeds to send my public entries to the inboxes of everyone who follows me. My node knows who follows me and thus can just send the public entry to each of those inboxes. Perhaps there will be a scaling problem in the future.

Later I write a friends-only entry about how much I hate the movie The Room (2003). Tommy Wiseau can't see it because I'm not friends with Tommy Wiseau, but this entry is sent to Steph's inbox, and she can see it because she's my friend.

When Steph logs into her node she'll see her stream and my public entry and my friends-only entry will be on her stream. She should also see that I've liked and commented her entry.

Scenario Summary

All actions from authors are routed through the inbox of the receiving authors by the node itself.

Nodes will store copies of entries because they receive them in the inbox.

Likes and comments on entries are all sent to the inbox of the author.

Public entries are sent to the inboxes of all followers of the author.

Friends-only entries are sent to the inboxes of all friends of the author.

Entries, likes, comments, follows are all sent to the inboxes of the authors that should be able to see them.

Project Parts

- Part 0: sign up a repo
- Part 1: 1/2 way implementation
- Part 2: Centralized implementation
- Part 3: Connected implementation
- Part 4: Connect with Groups
- Part 5: Finish it off!

Collaboration

- You may consult with other teams. The submission must be your own team's source code.
- You may work with 5 other students (groups of 6)
- Work will be submitted together
- Collaboration must be documented in the README.md file
- Any external source code must be referenced and documented in the README.md file.
- You make collaborate/consult with other groups in order to get your web services to integrate with each other

User Stories

- Identity
    - As an author, I want a consistent identity per node, so that URLs to me/my entries are predictable and don't stop working.
        - Note: This includes API links and web frontend links.
        - Note: This doesn't include authors/entries that are deleted.
    - As a node admin, I want to host multiple authors on my node, so I can have a friendly online community.

    - As an author, I want a public page with my profile information, so that I can link people to it.
    - As an author, I want to my (new, public) GitHub activity to be automatically turned into public entries, so everyone can see my GitHub activity too.
    - As an author, I want my profile page to show my public entries (most recent first), so they can decide if they want to follow me.
    - As an author, I want to be able to edit my profile: name, description, picture, and GitHub.
    - As an author, I want to be able to use my web browser to manage my profile, so I don't have to use a clunky API.
- Posting
    - As an author, I want to make entries, so I can share my thoughts and pictures with other local authors.
    - As an author, I want my node to send my entries to my remote followers and friends, so that remote authors following me can see them. ⊶ *Part 3-5 only*
    - As an author, I want to edit my entries locally, so that I'm not stuck with a typo on a popular entry.
        - That is, authors should not have to delete and re-create an entry to change the content.
    - As an author, I want my node to re-send entries I've edited to everywhere they were already sent, so that people don't keep seeing the old version. ⊶ *Part 3-5 only*
    - As an author, entries I make can be in CommonMark, so I can give my entries some basic formatting.
    - As an author, entries I make can be in simple plain text, because I don't always want all the formatting features of CommonMark.
    - As an author, entries I create can be images, so that I can share pictures and drawings.
    - As an author, entries I create that are in CommonMark can link to images, so that I can illustrate my entries.
    - As an author, I want to delete my own entries locally, so I can remove entries that are out of date or made by mistake.
    - As an author, I want my node to re-send entries I've deleted to everyone they were already sent, so I know remote users don't keep seeing my deleted entries forever. ⊶ *Part 3-5 only*
    - As an author, I want to be able to use my web-browser to manage/author my entries, so I don't have to use a clunky API.
        - We don't want authors to have to use some interface like django-rest-framework or Swagger.

    - As an author, other authors cannot modify my entries, so that I don't get impersonated.
- Reading
    - As an author, I want a "stream" which shows all the entries I should know about, so I don't have to switch between different pages.
        - As an author, I want my stream page to show me all the public entries my node knows about, so I can find new people to follow.
        - As an author, I want my stream page to show me all the unlisted and friends-only entries of all the authors I follow.
        - As an author, I want my stream page to show me the most recent version of an entry if it has been edited.
        - As an author, I want my stream page to **not** show me entries that have been deleted.
    - As an author, I want my "stream" page to be sorted with the most recent entries first.
- Visibility
    - As an author, I want to be able to make my entries "public", so that everyone can see them.
    - As an author, I want to be able to make my entries "unlisted," so that my followers see them, and anyone with the link can also see them.
    - As an author, I want to be able to make my entries "friends-only," so that I don't have to worry about people I don't know seeing them.
    - As an author, I want my friends to see my friends-only, unlisted, and public entries in their stream.
    - As an author, I want anyone following me to see my unlisted and public entries in their stream.
    - As an author, I want everyone to see my public entries in their stream.
    - As an author, I want everyone to be able to see my public and unlisted entries, if they have a link to it.
    - As an author, I don't anyone who isn't a friend to be able to see my friends-only entries and images, so I can feel safe about writing.
    - As an author, I don't want anyone except the node admin to see my deleted entries.
    - As an author, entries I create should always be visible to me until they are deleted, so I can find them to edit them or review them or get the link or whatever I want to do with them.
- Sharing
    - As a reader, I can get a link to a public or unlisted entry, so I can send it to my friends over email, discord, slack, etc.

    - As a node admin, I want to push images to users on other nodes, so that they are visible by users of other nodes. ⊶ *Part 3-5 only.*
    - As an author, I should be able to browse the public entries of everyone, so that I can see what's going on beyond authors I follow.
        - Note: this should include all local public entries and all public entries received in any inbox.
- Following/Friends
    - As an author, I want to follow local authors, so that I can see their entries.
    - As an author, I want to follow remote authors, so that I can see their entries. ⊶ *Part 3-5 only.*
    - As an author, I want to be able to approve or deny other authors following me, so that I don't get followed by people I don't like.
    - As an author, I want to know if I have "follow requests," so I can approve them.
    - As an author, I want to unfollow authors I am following, so that I don't have to see their entries anymore.
    - As an author, if I am following another author, and they are following me (only after both follow requests are approved), I want us to be considered friends, so that they can see my friends-only entries.
    - As an author, I want to unfriend other authors by unfollowing them, so that they can no longer see my friends-only entries.
    - As an author, my node will know about my followers, who I am following, and my friends, so that I don't have to keep track of it myself.
- Comments/Likes
    - As an author, I want to comment on entries that I can access, so I can make a witty reply.
    - As an author, I want to like entries that I can access, so I can show my appreciation.
    - As an author, I want to like comments that I can access, so I can show my appreciation.
    - As an author, when someone sends me a public entry I want to see the likes, so I can tell if it's good or not.
    - As an author, comments on my friends-only entries are visible only to my friends and the comment's author.
- node Management
    - As a node admin, images can be hosted on my node, so that my users can use them in their CommonMark entries.
    - As a node admin, I want to be able to add, modify, and delete authors, to fix problems or remove unwanted users.

- As a node admin, I want to be able to allow users to sign-up but require my approval to complete sign-up and use my node, so that I can prevent unwanted users and spambots.
- As a node admin, I want to be able to connect to remote nodes by entering only the URL of the remote node, a username, and a password, so that I don't have to edit code. ⊶ *Part 3-5 only.*
- As a node admin, I want a RESTful interface for most operations, so that I can connect to other nodes and allow my users to use alternate clients other than the web frontend.
- As a node admin, I want to be able to add nodes to share with. ⊶ *Part 3-5 only.*
- As a node admin, I want to be able to remove nodes and stop sharing with them. ⊶ *Part 3-5 only.*
- As a node admin, I can prevent nodes from connecting to my node if they don't have a valid username and password. ⊶ *Part 3-5 only.*
- As a node admin, node to node connections can be authenticated with HTTP Basic Auth, so that I don't have to deal with tokens. ⊶ *Part 3-5 only.*
- As a node admin, I can disable the node to node interfaces for connections that I no longer want, in case another node goes bad. ⊶ *Part 3-5 only.*
- As a node admin, I want everything to be stored in a well-indexed relational database, so that my website is snappy, and I can write SQL to fix things if I need to, make backups, etc...
  - Use Postgres on Heroku and SQLite for testing on your local machine.
  - Other DBaaS (e.g. Firebase) is forbidden.
- As a node admin, I don't want arrays to be stored in database fields, so that my node won't get slower over time.
- As a node admin, I don't want to have separate frontend and backend web servers, so I don't have to manage two web servers/services.
- As a node admin, I want deleted entries stay in the database and only be removed from the UI and API, so I can see what was deleted.
- As a node admin, I want my node's UI to only communicate with my nodes web server, so I can prevent XSS.
- As a node admin, I want the API objects (authors, entries, etc.) to be identified by their full URL, to prevent collisions with other node's numbering schemes. ⊶ *Part 3-5 only.*

Main Concepts

- Node
  - A single webserver plus a single database

    - Serves and stores everything needed for the entire app
    - Doesn't rely on additional web servers
    - Can connect to other nodes to exchange entries, likes, comments, follow requests...
    - Keeps track of the identities, entries, likes, comments, etc. of its authors
- Author
    - Makes entries
    - Following other authors
    - Can have followers
    - Makes friends
    - Likes entries
    - Comments on entries
    - A generally nice person
    - Can register with the admins approval
    - Can find other authors by using the public timeline
- Local Node
    - The node that an author is signed up for and uses to manage their entries, likes, comments, and profile.
- Remote Node
    - Any node that isn't the author's local node.
- Local Author
    - Another author on the same node as an author.
- Remote Author
    - Another author on a different node as an author.
- Node Admin
    - Manages a node
    - Allows people to sign up
    - Responsible for private data
    - Keeps deleted data forever
- Follower
    - An author
    - Someone who follows you (you are an author)
    - Your node will send entries to the follower's inbox
- Following
    - An author
    - Someone you are following
    - Their node will send entries to your inbox
- Friend
    - You follow them, and they follow you

- Restful service
    - The model of the service and its API
- Server
    - A single Django or flask web server
    - running on a single computer
    - Has its own unique database
- UI
    - The HTML/CSS/JS coated version user interface
    - Runs in the browser
    - Served from the web server of the node
    - Must only communicate with the node it was served from
- Entry
    - Like a Facebook post, tweet, etc.
    - It has a unique URL
    - The URL stays the same
    - The node keeps track of what inboxes have been notified
    - It can be edited
    - It can be deleted
    - It can be:
        - A picture
        - or A plain text
        - or A Markdown text
    - It can be:
        - public
        - or friends-only
        - or unlisted
        - or deleted
    - It can be liked
    - It can be commented on
    - Likes and comments are sent back to the author's inbox.
- Public Entry
    - This is an entry that will be sent to all of my follower's inboxes, to make sure their nodes know about it.
    - This is an entry that will show up in all the streams of all the authors on all the nodes where I have at least one follower.
    - This is an entry that will show up on my public profile page.
    - Anyone can see it.
    - Anyone can access it by URL.
    - Public entries can be liked by anyone.

    - Public entries can have comments from anyone.
- Unlisted Entry
    - This entry will **not** show up in the "stream" of authors that don't follow me.
    - This is an entry that will be sent to all my follower's inboxes.
    - Anyone can access it by URL.
    - Anyone can see the unlisted entry, if they already know about it.
    - Unlisted entries can be liked by anyone.
    - Unlisted entries can have comments from anyone.
- Friends-Only Entry
    - This is an entry that will be sent to all my friend's inboxes.
    - This is an entry that will show up in all my friend's streams.
    - This entry won't show up in anyone's stream unless they're my friend.
    - The URL can only be used by the author of the entry and admins.
    - Friends-Only entries can be liked by the author's friends.
    - Friends-Only entries can have comments from the author's friends.
    - This entry can be deleted.
- Deleted Entry
    - When Public/Unlisted/Friends-Only entry is deleted by the author, the node converts it to a Deleted Entry.
    - This is an entry that will be sent to all inboxes it was previously sent to.
    - This entry will not show up in anyone's stream.
    - This entry will not show up on any profile page.
    - This entry cannot be accessed at all except by the local node administrator.
    - This entry cannot be liked.
    - This entry cannot be commented on.
- Inbox
    - **An inbox is not something the user ever actually sees. It is an API endpoint *only*.**
    - This is what a READER or USER of the social network has. They follow authors, and the authors they follow send objects to their inbox.
    - This is something that only exists in the API. There is no special inbox in the User Interface, entries sent to a user's inbox are integrated into their stream.
    - This forms the backbone of the timeline of the social media user.
    - This receives likes and comments.
- Stream
    - The user interface showing date-sorted (most recent first) local, public entries combined with entries from everyone that user follows.
    - This is similar to "timeline" in old social media apps, "following" page, or "friends" page, or "newsfeed" in old Facebook.

- Remote
    - A node to node connection. Requests from another node. HTTP Basic Auth authenticated.
- Local
    - A local user accessing the REST API. Likely will use their cookie-auth, basic auth, or token auth. Local usually implies you check if the user should have access. For node local API access to the inbox should be limited to only that authenticated authors --- don't snoop!
- Profile Page
    - A page that shows information about me as well as my public entries.
- Push
    - When the node that has the information sends it other relevant nodes, without being requested.
- Pull
    - When the node that needs the information requests from the node that has it.
- FQID - Fully Qualified ID
    - The full URL of that object
    - It is completely unique
- Serial
    - Any string that that can be combined with other parts of a URL to form a FQID
    - Must **not** start with `http` or contain `:`
    - Not in any particular format (though it needs to be compatible with URLs)
    - Is **not** unique
        - Two different authors on two different nodes could be both serial #123
        - Two different entries from two different authors could be both serial c4f71e54-7b41-448e-a4fb-031f1a20007b

Frontend/API Visibility	Admin	Friend	Follower	Everyone
Public	control panel	link + stream	link + stream	link + stream
Unlisted	control panel	link + stream	link + stream	link
Friends Only	control panel	authenticated		

Deleted control panel

Database/Node2Node	Type	Author	Friend	Follower	Anyone
New Public	push	from	to `inbox`	to `inbox`	
New Unlisted	push		to `inbox`	to `inbox`	
New Friends Only	push		to `inbox`		
Deleted Public	push		to `inbox`	to `inbox`	to inboxes entry was sent to before
Deleted Unlisted	push		to `inbox`	to `inbox`	to inboxes entry was sent to before
Deleted Friends Only	push		to `inbox`		to inboxes entry was sent to before
Edited Public	push		to `inbox`	to `inbox`	to inboxes entry was sent to before

Edited Unlisted	push	from	to `inbox`	to `inbox`	to inboxes entry was sent to before
Edited Friends Only	push	from	to `inbox`		to inboxes entry was sent to before
Commented Public	push	to `inbox`	from	from	from
Commented Unlisted	push	to `inbox`	from	from	from
Commented Friends Only	push	to `inbox`	from		
Liked Public	push	to `inbox`	from	from	from
Liked Unlisted	push	to `inbox`	from	from	from
Liked Friends Only	push	to `inbox`	from		
Follow	push	to `inbox`			from
Follow-back (friend)	push	from		to `inbox`	
Unfollow	push		*not yet implemented*	*not yet implemented*	
View Public	pull	to `entry`	optional from	optional from	optional from
View Unlisted	pull	to `entry`	optional from	optional from	optional from
View Friends-Only	pull		*not yet implemented*		

View Deleted	pull			
View Following	pull	to `followers`	optional from	optional from

Notes on the above tables:

- This can work entirely on push from the author's server to another author's inbox.
- Yes, this means that only the local node (node where the entry came from) will have a complete list of comments/likes. Mastodon/Diaspora also have this problem.
- "Unfollow" the node-to-node inbox API is missing this functionality, however it should work locally.
    - When author1 on node1 unfollows author2 on node2: node1 should not add author2's posts to author1's stream, even though it will continue to receive them from node2.
- Yes, a node may have an out of date list of followers if a remote follower unfollows.
- "View Friends-Only" the API is missing this functionality.

Authentication

- Remote node authentication is global between nodes. It requires an admin to allow node to node access.
- Remote node authentication is done solely with HTTP basic auth
- Local auth uses your local auth mechanisms (cookies, tokens, basic auth).
- Local auth is between an author's UI and an author's node server.
    - It could be the web UI you make
    - It could also be another custom UI (like an android app)
- Local refers the RESTful API for authors on that node.
    - This is useful for frameworks like React, Vue, or Angular to get access to data, and enable a more client heavy UI.
    - Even if your frontend does not use them, endpoints marked [local] should be usable by a future Android/iOS client to achieve the same functionality as frontend.

Pagination

If something is paginated it has query options:

- `page_number` - how many pages of objects have been delivered
- `size` - how big is a page

- Page 4 of objects http://service/api/author/{AUTHOR_SERIAL}/entries/{ENTRY_SERIAL}/comments?page=4
- Page 4 of objects but 40 per page http://service/api/author/{AUTHOR_SERIAL}/entries/{ENTRY_SERIAL}/comments?page=4&size=40
- 1 based indexing. First page is 1.

Communication

- HTTP Methods (PUT POST GET DELETE) not explicitly listed are not allowed methods
    - Most HTTP methods are local API only, and provided for local node use.
    - Local node use means for use by the frontend to communicate with the backend it came from.
- You are free to add your own endpoints for local node use, but you must document and test them.

Who talks to Who

- Required: The backends talk to each other, mostly through POST requests to the `inbox` URLs for remote authors.
- Optional: The frontend talks to the backend for the same node.
- Forbidden: The frontend talks to a backend of another node.

Overview

Almost all node-to-node communication proceeds by the node where something (entry/like/comment/follow request) was created POSTing that thing that was created to the relevant authors inbox on a remote node.

Node-to-node (marked as "[remote]") request other than "POST to inbox" are rarely needed, but you should support them in case the remote node needs to check something.

Example (node-to-node API View)

The Frontend-to-Backend (also known as [local]) communication for this scenario is up to your team, this only describes the Backend-to-Backend communication between two different nodes.

1. I sign up for SocialDistribution on http://node1

2. node1 administrator approves my account.
3. Now an author object exists that represents me at http://node1/api/authors/555555555
4. I use the interface to make a follow request to Steph, who is node2/api/authors/777777777
5. My node POSTs the follow request to Steph's inbox at http://node2/api/authors/777777777/inbox
6. When Steph logs in, she sees my follow request and approves it.
7. Now Steph's node (node2) knows that I am following her.
8. Steph makes a public entry.
9. Steph's node (node2) sends Steph's new entry to my inbox with POST http://node1/api/authors/555555555/inbox.
10. I eventually see Steph's new entry, and click like on it.
11. My node sends the like to Steph's inbox with POST http://node2/api/authors/777777777/inbox

Note: When sending a follow request to a foreign author, you (as the sending author/node) do not need to await any form of confirmation from the receiving author.

For example, suppose author A sends a follow request to author B. Once the follow request is sent from A to B's inbox, A's node can assume that A is following B, even before author B accepts or denies the request.

If B later denies author A's request, then B's entries will never be sent to A's inbox. But from the perspective of A's node the acceptance or rejection of a follow request does not matter.

Just show them as followed!

IDs

Entries may be generated a UUID, or ID#, or whatever for internal database/model use. However, on the API the entry or author should always have a fully qualified URL as its ID, and you must always identify remote objects (authors, entries, likes, comments, ...) as their full URL, including the URL of the node they came from. This means you will need to look up authors and entries in your database by their full URL ID, even if they are local.

Consider the following scenario: if two nodes both have an author with primary key 2, your node should never be able to be confused about which node's author the database is referring to, because all entries/likes/comments/follows from that author contain the author's full URL.

Consider the following scenario #2:
`http://node1/api/authors/0192019f-b832-74b2-b2c4-f7aadc972cb2` is Jane, and

`http://node2/api/authors/0192019f-b832-74b2-b2c4-f7aadc972cb2` is John. Jane and John still need to be able to like/comment/follow with each other. That means, your database should not connect models using the field containing `0192019f-b832-74b2-b2c4-f7aadc972cb2`, but instead, the field containing `http://node1/api/authors/0192019f-b832-74b2-b2c4-f7aadc972cb2`.

Hint: In Django, set `unique=True` on the field. Then use `models.ForeignKey` with source and destination field names to relate the two models (join the two tables).

API Objects

Example Author Objects

```
{
    // Author object must always have type author
    "type":"author",
    // The full API URL for the author
    "id":"http://nodeaaaa/api/authors/111",
    // The full API URL for the author's node
    "host":"http://nodeaaaa/api/",
    // How the user would like the name to be displayed
    "displayName":"Greg Johnson",
    // URL of the user's github
    "github": "http://github.com/gjohnson",
    // URL of the user's profile image (external image in this
example)
    "profileImage": "https://i.imgur.com/k7XVwpB.jpeg"
    // URL of the user's HTML profile page
    // It could include an id number/uuid or not
    "web": "http://nodeaaaa/authors/greg"
}
```

```
{
    "type":"author",
    "id":"http://nodebbbb/api/authors/222",
    "host":"http://nodebbb/api/",
    "displayName":"Lara Croft",
    "github": "http://github.com/laracroft",
    // This author used an image from one of their entries
    "profileImage":
"http://nodebbb/api/authors/222/entries/217/image"
    // URL of the user's HTML profile page
```

```
    // It could include an id number/uuid or not
    "web":"http://nodebbb/authors/222",
}
```

Example Follow Request Object

```
{
    "type": "follow",
    "summary":"Greg wants to follow Lara",
    // state should be requesting, accepted or rejected
    "actor":{
        "type":"author",
        "id":"http://nodeaaaa/api/authors/111",
        "host":"http://nodeaaaa/api/",
        "displayName":"Greg Johnson",
        "github": "http://github.com/gjohnson",
        "profileImage": "https://i.imgur.com/k7XVwpB.jpeg",
        // URL of the user's HTML profile page
        "web": "http://nodeaaaa/authors/greg"
    },
    "object":{
        "type":"author",
        "id":"http://nodebbbb/api/authors/222",
        "host":"http://nodebbbb/api/",
        "displayName":"Lara Croft",
        // URL of the user's HTML profile page
        "web":"http://nodebbbb/authors/222",
        "github": "http://github.com/laracroft",
        "profileImage":
"http://nodebbbb/api/authors/222/entries/217/image"
    }
}
```

Example Entry Object

- Public entry

```
{
    "type":"entry",
    // title of an entry
    "title":"An entry title about an entry about web dev",
```

```
    // id of the entry
    // must be the original URL on the node the entry came from
    "id":"http://nodebbbb/api/authors/222/entries/249",
    // URL of the user's HTML profile page
    "web": "http://nodebbbb/authors/222/entries/293",
    // a brief description of the entry
    "description":"This entry discusses stuff -- brief",
    // The content type of the entry
    // assume either
    // text/markdown -- common mark
    // text/plain -- UTF-8
    // application/base64 # this an image that is neither a jpeg or
png
    // image/png;base64 # this is an png -- images are ENTRIES. So
you might have a user make 2 entries: an image entry then a text
entry that uses the image entry!
    // image/jpeg;base64 # this is an jpeg
    // for HTML you will want to strip tags before displaying
    "contentType":"text/plain",
    "content":"Þā wæs on burgum Bēowulf Scyldinga, lēof lēod-cyning,
longe þrāge folcum gefrǣge (fæder ellor hwearf, aldor of earde), oð
þæt him eft onwōc hēah Healfdene; hēold þenden lifde, gamol and
gūð-rēow, glæde Scyldingas. Þǣm fēower bearn forð-gerīmed in worold
wōcun, weoroda rǣswan, Heorogār and Hrōðgār and Hālga til; hy ̄rde ic,
þat Elan cwēn Ongenþēowes wæs Heaðoscilfinges heals-gebedde. Þā wæs
Hrōðgāre here-spēd gyfen, wīges weorð-mynd, þæt him his wine-māgas
georne hy ̄rdon, oð þæt sēo geogoð gewēox, mago-driht micel. Him on
mōd bearn, þæt heal-reced hātan wolde, medo-ærn micel men gewyrcean,
þone yldo bearn ǣfre gefrūnon, and þǣr on innan eall gedǣlan geongum
and ealdum, swylc him god sealde, būton folc-scare and feorum gumena.
Þā ic wīde gefrægn weorc gebannan manigre mǣgðe geond þisne
middan-geard, folc-stede frætwan. Him on fyrste gelomp ǣdre mid
yldum, þæt hit wearð eal gearo, heal-ærna mǣst; scōp him Heort naman,
sē þe his wordes geweald wīde hæfde. Hē bēot ne ālēh, bēagas dǣlde,
sinc æt symle. Sele hlīfade hēah and horn-gēap: heaðo-wylma bād,
lāðan līges; ne wæs hit lenge þā gēn þæt se ecg-hete āðum-swerian 85
æfter wæl-nīðe wæcnan scolde. Þā se ellen-gǣst earfoðlīce þrāge
geþolode, sē þe in þy ̄strum bād, þæt hē dōgora gehwām drēam gehy ̄rde
hlūdne in healle; þǣr wæs hearpan swēg, swutol sang scopes. Sægde sē
þe cūðe frum-sceaft fīra feorran reccan",
    // the author has an ID where by authors can be disambiguated
    "author":{
        "type":"author",
```

```
        // ID of the Author
        "id":"http://nodebbbb/api/authors/222",
        // the home host of the author
        "host":"http://nodebbbb/api/",
        // the display name of the author
        "displayName":"Lara Croft",
        // URL of the user's HTML profile page
        "web":"http://nodebbbb/authors/222",
        // HATEOAS url for Github API
        "github": "http://github.com/laracroft",
        "profileImage":
"http://nodebbbb/api/authors/222/entries/217/image"
    },
    // comments about the entry
    "comments":{
        "type":"comments",
        // this may or may not be the same as page for the entry,
        // depending if there's a separate URL to just see the
comments
        "web":"http://nodebbbb/authors/222/entries/249",
        "id":"http://nodebbbb/api/authors/222/entries/249/comments"
        // comments.page_number, comments.size, comments.count,
        // comments.src are only sent if:
        // * public
        // * unlisted
        // * friends-only and sending it to a friend
        // You should return ~ 5 comments per request.
        // should be sorted newest(first) to oldest(last)
        // this is to reduce API call counts
        // number of the first page of comments
        "page_number":1,
        // size of comment pages
        "size":5,
        // total number of comments for this entry
        "count": 1023,
        // the first page of comments
        "src":[
            {
                "type":"comment",
                "author":{
                    "type":"author",
                    "id":"http://nodeaaaa/api/authors/111",
                    "web":"http://nodeaaaa/authors/greg",
```

```
                    "host":"http://nodeaaaa/api/",
                    "displayName":"Greg Johnson",
                    "github": "http://github.com/gjohnson",
                    "profileImage":
"https://i.imgur.com/k7XVwpB.jpeg"
                },
                "comment":"Sick Olde English",
                "contentType":"text/markdown",
                // ISO 8601 TIMESTAMP
                "published":"2015-03-09T13:07:04+00:00",
                // ID of the Comment
                "id":"http://nodeaaaa/api/authors/111/commented/130",
                "entry":
"http://nodebbbb/api/authors/222/entries/249",
                // this may or may not be the same as page for the
entry,
                // depending if there's a separate URL to just see
the one comment in html
                "web": "http://nodebbbb/authors/222/entries/249"
                // it could also be something like
                //
"web":"http://nodeaaaa/api/authors/greg/comments/130"
                // likes on the comment, not to be confused with
likes on the entry
                "likes": {
                    "type": "likes",
                    "id":
"http://nodeaaaa/api/authors/111/commented/130/likes",
                    // in this example nodebbbb has a html page just
for the likes
                    "web":
"http://nodeaaaa/authors/greg/comments/130/likes"
                    "page_number": 1,
                    "size": 50,
                    "count": 0,
                    "src": [],
                },
            }
        ]
    },
    // likes on the entry
    "likes":{
        "type":"likes",
```

```
        // this may or may not be the same as page for the entry,
        // depending if there's a separate URL to just see the
comments
        "web":"http://nodeaaaa/authors/222/entries/249"
        "id":"http://nodeaaaa/api/authors/222/entries/249/likes"
        // likes.page, likes.size, likes.count,
        // likes.src should be sent for public and unlisted entries
        // in order to reduce API calls
        // You should return ~ 5 likes per request.
        // should be sorted newest(first) to oldest(last)
        // this is to reduce API call counts
        // number of the first page of likes
        "page_number":1,
        // size of a page of likes
        "size":50,
        // total number of likes
        "count": 9001,
        // the first page of likes
        "src":[
            {
                "type":"like",
                "author":{
                    "type":"author",
                    "id":"http://nodeaaaa/api/authors/111",
                    "web":"http://nodeaaaa/authors/greg",
                    "host":"http://nodeaaaa/api/",
                    "displayName":"Greg Johnson",
                    "github": "http://github.com/gjohnson",
                    "profileImage":
"https://i.imgur.com/k7XVwpB.jpeg"
                },
                // ISO 8601 TIMESTAMP
                "published":"2015-03-09T13:07:04+00:00",
                // ID of the Comment (UUID)
                "id":"http://nodeaaaa/api/authors/111/liked/166",
                // this should be the object they liked
                "object": "http://nodebbbb/authors/222/entries/249"
            }
        ]
    },
    // ISO 8601 TIMESTAMP
    "published":"2015-03-09T13:07:04+00:00",
    // visibility ["PUBLIC","FRIENDS","UNLISTED","DELETED"]
```

```
    "visibility":"PUBLIC"
    // for visibility PUBLIC means it is open to the wild web
    // FRIENDS means if we're friends I can see the entry
    // FRIENDS should've already been sent the entry so they don't
need thi
    // "DELETED" should never show up in the restful API or
frontend, but will need to be marked in the database
}
```

- Example, Friends-Only entry:

```
{
    "type":"entry",
    "title":"DID YOU READ MY ENTRY YET?",
    "id": "http://nodebbbb/api/authors/222/entries/293",
    // The frontend URL of this entry
    "web": "http://nodebbbb/authors/222/entries/293",
    "description":"Whatever",
    "contentType":"text/plain",
    "content":"Are you even reading my entries Arjun?",
    "author":{
        "type":"author",
        "id":"http://nodebbbb/api/authors/222",
        "host":"http://nodebbbb/api/",
        "displayName":"Lara Croft",
        "web":"http://nodebbbb/authors/222",
        "github": "http://github.com/laracroft",
        "profileImage": "https://i.imgur.com/k7XVwpB.jpeg"
    },
    "comments": {
        "type": "comments",
        "id": "http://nodebbbb/api/authors/222/entries/293/comments",
        // in this example nodebbbb has a html page just for the
comments
        "web": "http://nodebbbb/authors/222/entries/293/comments",
        "page_number": 1,
        "size": 5,
        "count": 0,
        "src": [],
    },
    "likes": {
        "type": "likes",
```

```
        "id":
"http://127.0.0.1:5454/api/authors/222/entries/293/likes",
        // in this example nodebbbb has a html page just for the
likes
        "web": "http://nodebbbb/authors/222/entries/293/likes"
        "page_number": 1,
        "size": 50,
        "count": 0,
        "src": [],
    },
    "published":"2015-03-09T13:07:04+00:00",
    "visibility":"FRIENDS"
}
```

Example Entries Object

```
{
    "type":"entries",
    // page number we're on (counting from 1)
    "page_number":23,
    // size of a page of entries
    "size":10,
    // total number of entries
    "count": 9001,
    // the first page of entries
    "src":[
        { "type":"entry", /* ... the rest of the entry object */ },
        { "type":"entry", /* ... the rest of the entry object */ },
        { "type":"entry", /* ... the rest of the entry object */ },
        { "type":"entry", /* ... the rest of the entry object */ },
        { "type":"entry", /* ... the rest of the entry object */ },
        { "type":"entry", /* ... the rest of the entry object */ },
        { "type":"entry", /* ... the rest of the entry object */ },
        { "type":"entry", /* ... the rest of the entry object */ },
        { "type":"entry", /* ... the rest of the entry object */ },
        { "type":"entry", /* ... the rest of the entry object */ },
    ]
}
```

Example Comment Object

```
{
```

```
    "type":"comment",
    "author":{
        "type":"author",
        "id":"http://nodeaaaa/api/authors/111",
        "web":"http://nodeaaaa/authors/greg",
        "host":"http://nodeaaaa/api/",
        "displayName":"Greg Johnson",
        "github": "http://github.com/gjohnson",
        "profileImage": "https://i.imgur.com/k7XVwpB.jpeg"
    },
    "comment":"Sick Olde English",
    "contentType":"text/markdown",
    // ISO 8601 TIMESTAMP
    "published":"2015-03-09T13:07:04+00:00",
    // ID of the Comment
    "id": "http://nodeaaaa/api/authors/111/commented/130",
    "entry": "http://nodebbbb/api/authors/222/entries/249",
    // likes on the comment
    "likes":{
        "type":"likes",
        // this may or may not be the same as page for the entry
        // this may or may not be the same as page for the comment
        // depending if there's a separate URL to just see the
comments
        "web":"http://nodeaaaa/authors/222/entries/249"
        "id":"http://nodeaaaa/api/authors/111/commented/130/likes"
        // likes.page, likes.size, likes.count,
        // likes.src should be sent for comments on public and
unlisted entries
        // in order to reduce API calls
        // You should return ~ 5 likes per request.
        // should be sorted newest(first) to oldest(last)
        // this is to reduce API call counts
        // number of the first page of likes
        "page_number":1,
        // size of a page of likes
        "size":50,
        // total number of likes
        "count": 9001,
        // the first page of likes
        "src":[
            {
                "type":"like",
```

```
                "author":{
                    "type":"author",
                    "id":"http://nodebbbb/api/authors/222",
                    "host":"http://nodebbbb/api/",
                    "displayName":"Lara Croft",
                    "web":"http://nodebbbb/authors/222",
                    "github": "http://github.com/laracroft",
                    "profileImage":
"http://nodebbbb/api/authors/222/entries/217/image"
                },
                // ISO 8601 TIMESTAMP
                "published":"2015-03-09T13:07:04+00:00",
                // ID of the Comment (UUID)
                "id": "http://nodeaaaa/api/authors/222/liked/255",
                "object":
"http://nodeaaaa/api/authors/111/commented/130"
            }
        ]
    },
}
```

Example Comments Object

```
{
    "type":"comments",
    // this may or may not be the same as page for the entry,
    // depending if there's a separate URL to just see the comments
    "web":"http://nodebbbb/authors/222/entries/249",
    "id":"http://nodebbbb/api/authors/222/entries/249/comments"
    // comments.page, comments.size, comments.count,
    // comments.src are only sent if:
    // * public
    // * unlisted
    // * friends-only and sending it to a friend
    // You should return ~ 5 comments per entry.
    // should be sorted newest(first) to oldest(last)
    // this is to reduce API call counts
    // number of the first page of comments
    "page_number":1,
    // size of comment pages
    "size":5,
    // total number of comments for this entry
```

```
    "count": 1023,
    // the first page of comments
    "src":[
        {
            "type":"comment",
            "author":{
                "type":"author",
                "id":"http://nodeaaaa/api/authors/111",
                "web":"http://nodeaaaa/authors/greg",
                "host":"http://nodeaaaa/api/",
                "displayName":"Greg Johnson",
                "github": "http://github.com/gjohnson",
                "profileImage": "https://i.imgur.com/k7XVwpB.jpeg"
            },
            "comment":"Sick Olde English",
            "contentType":"text/markdown",
            // ISO 8601 TIMESTAMP
            "published":"2015-03-09T13:07:04+00:00",
            // ID of the Comment
            "id":"http://nodeaaaa/api/authors/111/commented/130",
            "entry": "http://nodebbbb/api/authors/222/entries/249",
            // this may or may not be the same as page for the
entry,
            // depending if there's a separate URL to just see the
one comment in html
            "web": "http://nodebbbb/authors/222/entries/249"
            // it could also be something like
            // "web":"http://nodeaaaa/api/authors/greg/comments/130"
            // likes on the comment, not to be confused with likes
on the entry
            "likes": {
                "type": "likes",
                "id":
"http://nodeaaaa/api/authors/111/commented/130/likes",
                // in this example nodebbbb has a html page just for
the likes
                "web":
"http://nodeaaaa/authors/greg/comments/130/likes"
                "page_number": 1,
                "size": 50,
                "count": 0,
                "src": [],
            },
```

```
        }
    ]
}
```

Example Like Object

```
{
    "type":"like",
    "author":{
        "type":"author",
        "id":"http://nodeaaaa/api/authors/111",
        "web":"http://nodeaaaa/authors/greg",
        "host":"http://nodeaaaa/api/",
        "displayName":"Greg Johnson",
        "github": "http://github.com/gjohnson",
        "profileImage": "https://i.imgur.com/k7XVwpB.jpeg"
    },
    // ISO 8601 TIMESTAMP
    "published":"2015-03-09T13:07:04+00:00",
    "id":"http://nodeaaaa/api/authors/111/liked/166",
    // ID of the Comment (UUID)
    "object": "http://nodebbbb/api/authors/222/entries/249"
}
```

```
{
    "type":"like",
    "author":{
        "type":"author",
        "id":"http://nodebbbb/api/authors/222",
        "host":"http://nodebbbb/api/",
        "displayName":"Lara Croft",
        "web":"http://nodebbbb/authors/222",
        "github": "http://github.com/laracroft",
        "profileImage":
"http://nodebbbb/api/authors/222/entries/217/image"
    },
    // ISO 8601 TIMESTAMP
    "published":"2015-03-09T13:07:04+00:00",
    // ID of the Comment (UUID)
    "id": "http://nodeaaaa/api/authors/222/liked/255",
    "object": "http://nodeaaaa/api/authors/111/commented/130"
}
```

Example Likes Object

```
{
    "type":"likes",
    // this may or may not be the same as page for the entry,
    // depending if there's a separate URL to just see the comments
    "web":"http://nodeaaaa/authors/222/entries/249"
    "id":"http://nodeaaaa/api/authors/222/entries/249/likes"
    // likes.page, likes.size, likes.count,
    // likes.src should be sent for public and unlisted entries
    // in order to reduce API calls
    // You should return ~ 5 likes per entry.
    // should be sorted newest(first) to oldest(last)
    // this is to reduce API call counts
    // number of the first page of likes
    "page_number":1,
    // size of a page of likes
    "size":50,
    // total number of likes
    "count": 9001,
    // the first page of likes
    "src":[
        {
            "type":"like",
            "author":{
                "type":"author",
                "id":"http://nodeaaaa/api/authors/111",
                "web":"http://nodeaaaa/authors/greg",
                "host":"http://nodeaaaa/api/",
                "displayName":"Greg Johnson",
                "github": "http://github.com/gjohnson",
                "profileImage": "https://i.imgur.com/k7XVwpB.jpeg"
            },
            // ISO 8601 TIMESTAMP
            "published":"2015-03-09T13:07:04+00:00",
            // ID of the Comment (UUID)
            "id":"http://nodeaaaa/api/authors/111/liked/166",
            // this should be the object they liked
            "object": "http://nodebbbb/authors/222/entries/249"
        }
    ]
```

```
}
```

```
{
    "type": "likes",
    "id": "http://nodeaaaa/api/authors/111/commented/130/likes",
    // in this example nodebbbb has a html page just for the likes
    "web": "http://nodeaaaa/authors/greg/comments/130/likes"
    "page_number": 1,
    "size": 50,
    "count": 0,
    "src": [],
}
```

API Endpoints

Please see the definitions for FQID and serial above!

Authentication

- All "remote" endpoints need HTTP Basic authentication!
- Local endpoints need some type of authentication! (Your choice.)

Authors API

- URL: `://service/api/authors/`
    - GET [local, remote]: retrieve all profiles on the node (paginated)
        - page: how many pages
        - size: how big is a page
- Example query: GET `://service/api/authors?page=10&size=5`
    - Gets the 5 authors, authors 45 to 49.
- Example: GET `http://nodeaaaa/api/authors/`

```
{
    "type": "authors",
    "authors":[
        {
            "type":"author",
            "id":"http://nodeaaaa/api/authors/111",
            "host":"http://nodeaaaa/api/",
            "displayName":"Greg Johnson",
```

```
            "github": "http://github.com/gjohnson",
            "profileImage": "https://i.imgur.com/k7XVwpB.jpeg",
            "web": "http://nodeaaaa/authors/greg"
        },
        {
            // A second author object...
        },
        {
            // A third author object...
        }
    ]
}
```

Single Author API

- URL: `://service/api/authors/{AUTHOR_SERIAL}/`
    - GET [local]: retrieve `AUTHOR_SERIAL`'s profile
    - PUT [local]: update `AUTHOR_SERIAL`'s profile
- URL: `://service/api/authors/{AUTHOR_FQID}/`
    - GET [remote]: retrieve `AUTHOR_FQID`'s profile
- Example GET `http://nodeaaaa/api/authors/111`:

```
{
    // must be type author
    "type":"author",
    // should match the URL for get
    // must match the logged in author for PUT
    "id":"http://nodeaaaa/api/authors/111",
    // should match the host we're making the request to
    "host":"http://nodeaaaa/api/",
    // get/update the display name of the author
    "displayName":"Lara Croft",
    // get/update the github of the author
    "github": "http://github.com/gjohnson",
    // get/update the profile picture of the author
    "profileImage": "https://i.imgur.com/k7XVwpB.jpeg",
    // get the HTML profile page
    "web": "http://nodeaaaa/authors/greg"
}
```

Following API

- This API is only for a local author managing who they follow through the API. All requests must be authenticated for AUTHOR_SERIAL.
- URL: `://service/api/authors/{AUTHOR_SERIAL}/following` (must be authenticated)
    - GET [local]: get a list of authors who `AUTHOR_SERIAL` is following
- URL: `://service/api/authors/{AUTHOR_SERIAL}/following/{FOREIGN_AUTHOR_FQID}` (must be authenticated)
    - Note: foreign author ID should be a percent encoded URL of the foreign author. An example URL would be:
        - `http://example-node-1/api/authors/178aba49-ca39-4741-b227-f40d072b1222/following/http%3A%2F%2Fexample-node-2%2Fauthors%2F5f57808f-0bc9-4b3d-bdd1-bb07c976d12d`
    - DELETE [local]: `AUTHOR_SERIAL` unfollows `FOREIGN_AUTHOR_FQID` (must be authenticated)
    - GET [local] `AUTHOR_SERIAL` checks if they are following `FOREIGN_AUTHOR_FQID`
    - PUT [local] `AUTHOR_SERIAL` generates a follow request (if none exists) for `FOREIGN_AUTHOR_FQID`. If foreign author is on a different node, author's node should generate a follow request and POST it to foreign author's inbox API endpoint.

Followers API

- All requests must be authenticated for AUTHOR_SERIAL [local] or FOREIGN_AUTHOR's server must be authenticated [remote].
- URL: `://service/api/authors/{AUTHOR_SERIAL}/followers/{FOREIGN_AUTHOR_FQID}` (local author is being followed)
    - Note: foreign author ID should be a percent encoded URL of the foreign author. An example URL would be:
        - `http://example-node-1/api/authors/178aba49-ca39-4741-b227-f40d072b1222/followers/http%3A%2F%2Fexample-node-2%2Fauthors%2F5f57808f-0bc9-4b3d-bdd1-bb07c976d12d`
    - PUT [local]: accept `FOREIGN_AUTHOR_FQID` as a follower of `AUTHOR_SERIAL` (must be authenticated)
        - Used by `AUTHOR_SERIAL` to accept follow request.

        - Must return 404 if there is no matching follow request.
    - DELETE [local]: remove `FOREIGN_AUTHOR_FQID` as a follower of `AUTHOR_SERIAL` (must be authenticated)
        - REQUIRED: Used by `AUTHOR_SERIAL` to deny follow request.
        - OPTIONAL: Used to revoke follow request once it's already been accepted. (There's no user story for revoking follow requests after they're accepted.)
        - Must return 404 if there is no matching follow request or follower.
    - GET [local, remote] check if `FOREIGN_AUTHOR_FQID` is a follower of `AUTHOR_SERIAL`
        - Should return 404 if they're not
        - This is how you can check if follow request is accepted
        - Use by remote server is optional: there's no need to know whether a follow request was accepted, but it can improve the UI.
        - Implementation of the GET endpoint is required: only a different node actually using it is optional.
- Example: GET `http://nodeaaa/api/authors/111/followers`

```
{
    "type": "followers",
    "followers":[
        {
            "type":"author",
            "id":"http://nodebbbb/api/authors/222",
            "host":"http://nodebbbb/api/",
            "displayName":"Lara Croft",
            "web":"http://nodebbbb/authors/222",
            "github": "http://github.com/laracroft",
            "profileImage":
"http://nodebbbb/api/authors/222/entries/217/image"
        },
        {
            // Second follower author object
        },
        {
            // Third follower author object
        }
    ]
}
```

- Example: GET `http://nodeaaa/api/authors/111/followers/http%3A%2F%2Fnodebbbb%2Fapi%2Fauthors%2F222`

```
// if laura follows greg, otherwise 404
{
        "type":"author",
        "id":"http://nodebbbb/api/authors/222",
        "host":"http://nodebbbb/api/",
        "displayName":"Lara Croft",
        "web":"http://nodebbbb/authors/222",
        "github": "http://github.com/laracroft",
        "profileImage":
"http://nodebbbb/api/authors/222/entries/217/image"
}
```

Follow Request API

- URL: `://service/api/authors/{AUTHOR_SERIAL}/follow_requests`
  - `GET` [local]:
    - Used by `AUTHOR_SERIAL` to check who wants to follow them through the API
- URL: `://service/api/authors/{AUTHOR_SERIAL}/inbox`
  - `POST` [remote]: send a follow request to `AUTHOR_SERIAL`
    - `AUTHOR_SERIAL` will be the `object` below
- When author 1 (actor) tries to follow author 2 (object), author 1's node send the follow request to author 2's node.
- If the author 2 accepts the Follow Request then author 1 is following author 2.
  - If author 2 is also already following author 1, then they are now friends.
- Author2's node does not need to tell Author1's node if follow was accepted or rejected.
- Sent to inbox of "object"
- See the [follow request object]

```
{
    "type": "follow",
    "summary":"actor wants to follow object",
    "actor":{
        "type":"author",
        // The rest of the author object for the author who wants to
follow
```

```
    },
    "object":{
        "type":"author",
        // The rest of the author object for the author they want to follow
    }
}
```

Entries API

- URL: `://service/api/authors/{AUTHOR_SERIAL}/entries/{ENTRY_SERIAL}`
    - GET [local, remote] get the public entry whose serial is `ENTRY_SERIAL`
        - friends-only entries: must be authenticated
    - DELETE [local] remove a
        - local entries: must be authenticated locally as the author
    - PUT [local] update an entry
        - local entries: must be authenticated locally as the author
- URL: `://service/api/entries/{ENTRY_FQID}`
    - GET [local] get the public entry whose URL is `ENTRY_FQID`
        - friends-only entries: must be authenticated
- Creation URL ://service/api/authors/{AUTHOR_SERIAL}/entries/
    - GET [local, remote] get the recent entries from author `AUTHOR_SERIAL` (paginated)
        - Not authenticated: only public entries.
        - Authenticated locally as author: all entries.
        - Authenticated locally as follower of author: public + unlisted entries.
        - Authenticated locally as friend of author: all entries.
        - Authenticated as remote node: This probably should not happen. Remember, the way remote node becomes aware of local entries is by local node pushing those entries to inbox, not by remote node pulling.
    - POST [local] create a new entry but generate a new `ID`
        - Authenticated locally as author
- Be aware that Entries can be images that need base64 decoding.
    - entries can also hyperlink to images that are public
- Uses the same format as the entry object
- For [local] service, fields included may differ. For example, when first creating the entry, there's no reason to have likes, comments, etc. because it doesn't exist yet. Be sure to document this!

Image Entries

Image Entries are just entries that are images. But they are encoded as base64 data. You can inline an image entry using a data URL, or you can use this shortcut to get the image if authenticated to see it.

- URL: `://service/api/authors/{AUTHOR_SERIAL}/entries/{ENTRY_SERIAL}/image`
  - GET [local, remote] get the public entry converted to binary as an image
  - return 404 if not an image
- URL: `://service/api/entries/{ENTRY_FQID}/image`
  - GET [local, remote] get the public entry converted to binary as an image
  - return 404 if not an image
- This end point decodes image entries as images. This allows the use of image tags in Markdown.
- You can use this to proxy or cache images.

Comments API

- URL: `://service/api/authors/{AUTHOR_SERIAL}/inbox`
  - `POST` [remote]: comment on an entry by `AUTHOR_SERIAL`
  - Body is a comment object
- URL: `://service/api/authors/{AUTHOR_SERIAL}/entries/{ENTRY_SERIAL}/comments`
  - `GET` [local, remote]: the comments on the entry
  - Body is a "comments" object
- URL: `://service/api/entries/{ENTRY_FQID}/comments`
  - `GET` [local, remote]: the comments on the entry (that our server knows about)
  - Body is a "comments" object
- URL: `://service/api/authors/{AUTHOR_SERIAL}/entries/{ENTRY_SERIAL}/comments/{REMOTE_COMMENT_FQID}`
  - GET [local, remote] get the comment
- Example: GET `http://nodebbbb/api/authors/222/entries/249/comments/http%3A%2F%2Fnodeaaaa%2Fapi%2Fauthors%2F111%2Fcommented%2F130`:

```
{
    "type":"comment",
```

```
    "author":{
        "type":"author",
        "id":"http://nodeaaaa/api/authors/111",
        "web":"http://nodeaaaa/authors/greg",
        "host":"http://nodeaaaa/api/",
        "displayName":"Greg Johnson",
        "github": "http://github.com/gjohnson",
        "profileImage": "https://i.imgur.com/k7XVwpB.jpeg"
    },
    "comment":"Sick Olde English",
    "contentType":"text/markdown",
    // ISO 8601 TIMESTAMP
    "published":"2015-03-09T13:07:04+00:00",
    // ID of the Comment
    "id": "http://nodeaaaa/api/authors/111/commented/130",
    "entry": "http://nodebbbb/api/authors/222/entries/249",
}
```

Commented API

- URL: `://service/api/authors/{AUTHOR_SERIAL}/commented`
    - GET [local, remote] get the list of comments author has made on:
        - [local] any entry
        - [remote] public and unlisted entries
        - paginated
    - POST [local] if you post an object of "type":"comment", it will add your comment to the entry whose `ID` is in the `entry` field
        - Then the node you posted it to is responsible for forwarding it to the correct inbox
- URL: `://service/api/authors/{AUTHOR_FQID}/commented`
    - GET [local] get the list of comments author has made on any entry (that local node knows about)
- URL: `://service/api/authors/{AUTHOR_SERIAL}/commented/{COMMENT_SERIAL}`
    - GET [local, remote] get this comment
- URL: `://service/api/commented/{COMMENT_FQID}`
    - GET [local] get this comment
- Example: GET `http://nodeaaaa/api/authors/111/comments`:

```
[
```

```
    {
        "type":"comment",
        "author":{
            "type":"author",
            "id":"http://nodeaaaa/api/authors/111",
            "web":"http://nodeaaaa/authors/greg",
            "host":"http://nodeaaaa/api/",
            "displayName":"Greg Johnson",
            "github": "http://github.com/gjohnson",
            "profileImage": "https://i.imgur.com/k7XVwpB.jpeg"
        },
        "comment":"Sick Olde English",
        "contentType":"text/markdown",
        // ISO 8601 TIMESTAMP
        "published":"2015-03-09T13:07:04+00:00",
        // ID of the Comment
        "id":"http://nodeaaaa/api/authors/111/commented/130",
        "entry": "http://nodebbbb/api/authors/222/entries/249",
        // this may or may not be the same as page for the entry,
        // depending if there's a separate URL to just see the one
comment in html
        "web": "http://nodebbbb/authors/222/entries/249"
        // it could also be something like
        // "web":"http://nodeaaaa/authors/greg/comments/130"
    }
]
```

- Example: GET `http://nodeaaaa/api/authors/111/commented/130`:

```
{
    "type":"comment",
    "author":{
        "type":"author",
        "id":"http://nodeaaaa/api/authors/111",
        "web":"http://nodeaaaa/authors/greg",
        "host":"http://nodeaaaa/api/",
        "displayName":"Greg Johnson",
        "github": "http://github.com/gjohnson",
        "profileImage": "https://i.imgur.com/k7XVwpB.jpeg"
    },
    "comment":"Sick Olde English",
    "contentType":"text/markdown",
```

```
    // ISO 8601 TIMESTAMP
    "published":"2015-03-09T13:07:04+00:00",
    // ID of the Comment
    "id":"http://nodeaaaa/api/authors/111/commented/130",
    "entry": "http://nodebbbb/api/authors/222/entries/249",
    // this may or may not be the same as page for the entry,
    // depending if there's a separate URL to just see the one
comment in html
    "web": "http://nodebbbb/authors/222/entries/249"
    // it could also be something like
    // "web":"http://nodeaaaa/authors/greg/comments/130"
}
```

Likes API

- URL: `://service/api/authors/{AUTHOR_SERIAL}/inbox`
  - `POST` [remote]: send a like object to `AUTHOR_SERIAL`
  - Body is like object
- URL: `://service/api/authors/{AUTHOR_SERIAL}/entries/{ENTRY_SERIAL}/likes`
  - "Who Liked This Entry"
  - `GET` [local, remote] a list of likes from other authors on `AUTHOR_SERIAL`'s entry `ENTRY_SERIAL`
  - Body is likes object
- URL: `://service/api/entries/{ENTRY_FQID}/likes`
  - "Who Liked This Entry"
  - `GET` [local] a list of likes from other authors on `AUTHOR_SERIAL`'s entry `ENTRY_SERIAL`
  - Body is likes object
- URL: `://service/api/authors/{AUTHOR_SERIAL}/entries/{ENTRY_SERIAL}/comments/{COMMENT_FQID}/likes`
  - "Who Liked This Comment"
  - `GET` [local, remote] a list of likes from other authors on `AUTHOR_SERIAL`'s entry `ENTRY_SERIAL` comment `COMMENT_FQID`
  - Body is likes object

Liked API

- URL: `://service/api/authors/{AUTHOR_SERIAL}/liked`
    - "Things Liked By Author"
    - `GET` [local, remote] a list of likes by AUTHOR_SERIAL
    - Body is likes object
- URL: `://service/api/authors/{AUTHOR_SERIAL}/liked/{LIKE_SERIAL}`
    - `GET` [local, remote] a single like
    - Body is like object
- URL: `://service/api/authors/{AUTHOR_FQID}/liked`
    - "Things Liked By Author"
    - `GET` [local] a list of likes by AUTHOR_FQID
    - Body is likes object
- URL: `://service/api/liked/{LIKE_FQID}`
    - `GET` [local] a single like
    - Body is like object

Other Local APIs

Local APIs, such as "stream", that aren't specified here, are up to your design. However, you must document and test them.

Inbox API

- The inbox receives all the new entries from who you follow, as well as "follow requests," likes, and comments you should be aware of.
- Remember: the inbox is not something on the UI or the API for local clients! The inbox is the way that a node becomes aware of entries, likes, comments, follow requests, that it should be aware of from other nodes.
- URL: ://service/api/authors/{AUTHOR_SERIAL}/inbox
    - POST [remote]: send an entry to the author
    - if the type is "entry" then add that entry to AUTHOR_SERIAL's inbox
    - if the type is "follow" then add that follow is added to AUTHOR_SERIAL's inbox to approve later
    - if the type is "Like" then add that like to AUTHOR_SERIAL's inbox
    - if the type is "comment" then add that comment to AUTHOR_SERIAL's inbox
- When sending/updating entries, body is an entry object
- When sending/updating comments, body is a comment object
- When sending/updating likes, body is a like object
- When sending/updating follow requests, body is a follow object

Requirements

- Implement the webservice as described in the user stories
- Provide a webservice interface that is restful
- Provide a web UI interface that is usable
- Prove your project by connecting with other teammate's clone of your project. (Parts 3-5)
- Prove your project by connecting with at least 2 other groups. (Parts 4-5)
- Show that you are making a serious effort to connect with at least 3 other groups. (Parts 4-5)
- Make a video demo of your blog (desktop-recorder is ok)
    - Your video may **not** be a part of your presentation.
- Make a presentation about your blog
- Follow the guidelines in the project spec for URLs and services
- Allow users to accept or reject follow requests
- Images get the same protection that entries get as they are POSTS
- Follow the API requirements in the following section
- Follow the Infrastructure and Software Requirements in the following section

API Requirements

When building your API, try to adhere to these rules for easy compatibility with other groups:

- REST API calls may be prefixed like: `http://service_address/api/authors/{AUTHOR_SERIAL}/entries/`
- Document your service address, port, hostname, prefix (if used), and the username/password for HTTP Basic Auth in your README so that HTTP clients can connect to your API.
- You **must** be compatible with the API specification and examples listed above in this document.
    - You **may** need to add some additional things for compatibility with other groups due to varying interpretations.
    - You **may** add additional endpoints and JSON keys as long as you provide the ones listed above.
    - If you follow something close to, but not compatible with the API specification in this document, then you will get something close to, but not exactly, a good grade.

Test Requirements

The REST API must be fully tested.

- Every API endpoint must be tested.
- Every API functionality must be tested.
- Every API method must be tested.
- Every user story must be tested at the API level.
- If a user story does not have an API, it must be tested by calling the Django code directly.
- Tests that don't actually test what they say they're testing do not count.

These would be system/acceptance tests if the project didn't have a frontend.

Testing the user stories at the API level should give you the tests you need for every API endpoint, every API functionality, and every API method. **Pretend you are writing tests to make sure a future Android client will work with your API.**

Not every user story has an API to test. For example, adding/removing nodes to connect with usually does not have an API. For these functions, please test them in whatever way is most convenient for your project, and then verify them using the API. You could write a test that adds a node to connect with by calling Python code or modifying the database directly, then use the API to check that the new node connection is working.

Frontend (Selenium, etc.) tests are not required. Code coverage (line coverage, statement coverage, branch coverage, MC/DC, etc.) is not required. Unit testing is not required.

Documentation Requirements

- Every API endpoint is documented.
    - When the API endpoint should be used
    - How the API endpoint should be used
    - Why the API endpoint should or should not be used
- Every API endpoint has multiple examples.
- Every JSON field in both request and response
    - Has a type
    - Has an example value
    - Has an explanation of its purpose (when to use it)
    - For the request
    - For the response
- Explanation of anything interesting about each endpoint
    - Whether it is paginated, ...

            - How to use the pagination, ...
- If you are using some automated documentation generation, you must augment that documentation with examples, types, explanations, when, what, how, why, why not...

Code Requirements

- Code is well organized and has good style
    - Readable
    - No smells such as "god files/classes/functions"
    - Each folder/file/class/function only does one thing
    - Comments add to the code
    - All names are meaningful
    - Meets style guide for relevant language
        - Python: PEP8
        - JS: https://github.com/rwaldron/idiomatic.js/ or https://github.com/standard/standard
        - HTML/CSS: https://google.github.io/styleguide/htmlcssguide.html
        - Browser/interpreter/transpiler/bundler should not have any warnings or errors
        - Passes W3 Validation
            - HTML: https://validator.w3.org/
            - CSS: https://jigsaw.w3.org/css-validator/
        - You may use the latest HTML/CSS/JS features if supported by Firefox. Check https://caniuse.com/ or MDN
        - Not using any obsolete or deprecated HTML/CSS/JS
            - Check MDN, they have a little red trash icon
    - Good indentation
    - Well organized
    - It should be very easy for a TA or instructor to find the all the code responsible for a user story

Tool Use Requirements

- All teammates are using Git and GitHub
    - Evidence of teammates using Git and GitHub to work together should be obvious
    - Working on code at the same time
    - Commit messages are helpful information for teammates and TA
    - Using Git and GitHub to communicate with each other
    - The repository (including branches and tags) is well-organized.

    - Commits/merges/PRs are small and there are a lot of them.
        - Small = less than 2 hours of code
        - PRs are not required

Infrastructure and Software Requirements

Failure to meet these requirements will result in a score of zero for project parts.

We strongly recommend that you make your frontend using HTML, CSS, JS. You can use Jinja templates (it comes with Django) or WebComponents.

- We will not be covering frontend frameworks such as React, Angular, Vue, Solid, Svelte... (there are dozens of these) in lecture or in lab, for a number of reasons:
    - We don't have time to cover this during the semester. It would require a second semester to cover these.
        - You will have to learn how to work with these frameworks yourself.
    - React is losing popularity.
        - In ten years, knowing React will not be valuable anymore, because everyone will redo their websites in something else. This already happened with jQuery. 10 years ago all the 404 students wanted to learn jQuery, and now people barely use it.
    - There is no standard frontend framework other than plain HTML/CSS/JS.
    - The fact that large businesses like Facebook, Google, and Walmart use them does not mean they are good for a small semester project with a small team.
    - All of the React-style (reactive paradigm) frameworks have major flaws that cannot be fixed, because the paradigm itself is flawed.
    - All of the React-style (reactive paradigm) frameworks and most of the other frameworks are much more difficult to debug than plain JS or JS with a few libraries. This is because they make it difficult or impossible to determine why a component is being rerendered. They make it difficult or impossible to trace the code backwards from an error to the source of the problem.
    - All of the component-based frameworks produce really bad HTML for the browser that is also difficult to debug or understand how it was produced.
    - All frameworks using a shadow DOM are difficult to debug because the DOM updates are applied all at once, instead of step-by-step.
    - All of the important contributions of React (components, shadow DOM, scoped CSS) have already been added to native HTML/CSS/JS, removing the need for React.
        - Once again, this already happened with jQuery. All the good parts of jQuery got added to native JS, and now people barely use it.

    - A strong knowledge of HTML/CSS/JS allows you to quickly learn any framework, while knowing one framework does not teach you much about HTML/CSS/JS.
    - It's very easy to make your own React-style frontend framework. There is nothing special about React. React's popularity comes from the fact that Facebook made it. Angular is popular because Google made it.
    - Any particular frontend framework may not be able to meet the project requirements.
- TAs will not help you with frontend frameworks.
- Every semester there are at least a few students who choose to use a frontend framework and then, as a result:
    - End up having to redo large amounts of work when the framework turns out to not meet the requirements of the project
    - End up not being able to get parts of their application working due to the complexity and difficulty of debugging the framework code
    - End up with a web app that is incredibly slow
    - End up needing custom API endpoints which aren't specified above, so they have to do extra work
    - End up fighting among themselves because only one teammember can understand the framework

If you want to use React or a similar frontend framework such as Angular, Vue, Solid, Svelte... (there are dozens of these), despite these warnings, then all team members must fill out the frontend framework form. We offer this option only because some students want the experience for their resumé. However, it involves extra work. If **all team members** do not fill out the frontend framework form agreeing to use a front end framework despite the extra work required and the above warnings then you are not allowed to use a frontend frameworks, and using them will result in a mark of zero.

Frontend Framework Consent Form

- Backend frameworks are limited to Django and Flask only. We suggest Django. Flask requires extra work.
- Server-side rendering is limited to Jinja templates (provided by Django), or another templating engine that can run in Django/Flask.
- Combined frontend-backend frameworks, and server-side component rendering is not allowed unless you can get it working in Django on Heroku.
    - Server side rendering requiring a node server is not allowed.
    - Anything requiring a node server is not allowed.

    - Next.js, ... and similar server-side frameworks are not allowed. The only allowed server-side frameworks are Django and Flask.
- You must use a single git repository on GitHub Classroom that contains everything.
    - Must have a `development` branch.
    - Must have a `production` branch.
    - Must **not** contain built (compiled, transpiled, bundled) or downloaded artifacts, including but not limited to:
        - No downloaded Python packages, `virtualenv` `venv` etc.
        - No built Python artifacts: `.pyc` files, `__pycache__` directories, etc.
        - No downloaded JS packages: `node_modules` etc.
        - No HTML/CSS/JS output produced by compiler/transpiler/bundler
        - No `esbuild`, `vite`, `rollup`, `webpack`, etc. output is included in the GitHub repository.
        - Failure to meet the above restrictions may result in a mark of zero.
    - Note: If you want to keep a copy at the end of the semester, do not use GitHub fork: clone it to your computer, and push it to a new GitHub repository (manual fork).
    - Force-push, rebase, and other git operations that remove history from GitHub are strictly forbidden, and you may receive a zero for project parts or be removed from your team.
- Your project must be compatible with Firefox browser.
    - Firefox will be used as the standard for marking. We will not check your code in multiple browsers to try to get it working.
- Your project must be hosted on an approved hosting solution.
    - Heroku
        - An entire node of your project must only require:
            - A single web "dyno"
            - with a single PostgreSQL add-on
        - Every teammate must be able to deploy their own project node to their own Heroku.
    - Individual hosting on approval: you must get approval in advance from the instructor.
        - You are responsible for any problems that come up as a result.
        - You are responsible for any surprise fees that come up as a result.
        - Often when people use some other hosting service that isn't Heroku they get shut off, blocked by firewalls, and/or hit with surprise fees.
        - This only applies on an individual basis: teammates must still use Heroku.
            - Project code must still be deployable by teammates to Heroku.
        - Database provider must be the same provider as your website hosting.

- Every teammate must be able to deploy their own node using the same code.
    - This means 6 nodes using the same project code but running as different (Heroku) servers.
- Use a single Django or Flask server per node.
    - A single Django or Flask server must serve *everything*:
        - Frontend (for browser) dynamic
        - Frontend (for browser) static content
            - Use whitenoise for static content.
        - Backend API
            - Use routes to distinguish.
        - Use Python 3.11+
    - Splitting frontend/backend hosting is strictly not allowed. Splitting static/dynamic is strictly not allowed.
        - They must be a part of the same Heroku + (Django or Flask) server.
        - Violation of this restriction will result in a mark of zero.
- Deployed nodes (Heroku): Must use a PostgreSQL database for all content storage.
- Local development nodes (your laptop): PostgreSQL or SQLite for all content storage.
    - SQLite is recommended.
- Everything that's not in a database files must be code managed by GitHub.
    - Code checked into git must be clean.
    - Cloud (e.g. Firebase), NoSQL, key-value, and file system storage are strictly forbidden.
- License your code properly (use an OSI approved license)
    - Put your name (or some representation of you like GeneralHuxFan768) on it!

Takeaways

- 1 Working Website
- 1 GitHub git repo
- 1 Presentation
    - May not include the video
- 1 OpenAPI ('Swagger') specification for your API
- 1 Video

Things that are allowed

- You may use React if you choose, but generally this is more difficult and takes more work so it is not recommended.

    - Django templates or lighter-weight frameworks like Vue are generally easier and take less work.
    - If you use React, you must still serve everything (including React JS and HTML) through the same Django/Flask server that is serving the API backend.

Teamwork Tips

- These optional. They are just advice. They usually lead to less conflict and a better end result.

The most successful teams:

- Do not use pull requests.
    - Pull requests are great in general, but they tend to cause problems because the project is very small and groups are also small.
- Use very, very few branches (e.g. production and staging).
    - Do not use author branches (or branches where a single author is the only one touching them).
- Commit, pull, push the code they're working on, to the same branch that everyone else is working on, at least once an hour.
    - Commit code that doesn't break the overall project.
    - Avoid your version being broken for more than an hour.
    - Do not stay up all night, change everything, and push a hundred changes all over the code base as one giant pull request.
- Divide teamwork by user story, instead of frontend vs backend.

Submission Instructions

- Submission will be by GitHub Classroom. Please follow the link on Canvas. Make sure you also submit a link on Canvas on time!

Marking

- Usually all teammates will share the same mark.
- An individual student's marks will be reduced if the instructor (or a TA in consultation with the instructor) finds that:
    - The student is not contributing or contributing significantly less than their teammates.

    - The individual student is significantly preventing their teammates from participating or contributing.
    - The individual student's is significantly misusing git/GitHub/Heroku/other software tools.
    - The individual student is engaging in significant "intellectual violence."
        - Intellectual violence is when one teammate uses their skill, knowledge, or experience, to intimidate or control the other teammate(s) rather than sharing and helping them learn.
    - The individual student is not communicating or only communicating very rarely with their team.
    - The individual student does not complete the peer feedback form (-1 mark for each part 1-4).
- Any concerns about teamwork must be brought to the instructor's (not the TA) attention *by email* well before the last lab.
    - It takes time to investigate these things.

Overall Marking

- Excellent 8 (A+): Clean code. Meets the requirements and adds extra polish. Everything is tested properly. Passes all tests. The API is documented in detail. The API is implemented as specified, and adds extra for compatibility. The UI meets all requirements, and has extra polish.
- Good 7 (B+): Code is mostly clean but has some rough spots. At most a couple of minor bugs. Almost all the requirements are met. Everything is tested. Passes almost all tests. Almost everything is documented. Almost all the API is implemented according to spec.
- Satisfactory 6 (C+): Code is low quality but working. Some bugs. Inconsistency. ¾ of the requirements are met. Almost everything is tested. Passes most tests. Most things are documented. API is implemented but doesn't meet spec.
- Unsatisfactory 4 (D): There are significant bugs and issues. ½ of the requirements are met. ½ the tests exist and pass. At least ½ of the documentation is present. At least ½ of the API exists and works. At least ½ of the UI exists and works. At least ½ of the API exists and works.
- Partial Attempt 1 (F): Poor quality. Large pieces of code missing, some pieces exist. Some requirements are met. Some of the UI exists. Some of the API exists.
- Failure 0 (F): Missing. Not attempted. Not complete enough to make an evaluation. Violated a restriction.

UI amount complete is marked by your TA testing user stories manually.

If a user story is partially complete, it will count the same as being incomplete.

Project Parts

Project Part 0: Group Formation

You must form a group with only students from your same lab section. You can have different lecture sections, but your lab section must be the same. Furthermore, everyone must attend their registered lab section.

- There can be maximum 6 students per team and a minimum of 5 students.
- All team members must be registered for and attending the same lab section.
- Your team name must be from the list of CSS colour names.
- Use the GitHub Classroom Link on Canvas
    - EVERY teammate should submit a link to your teams GitHub repo on Canvas.
- Add a license and a README. The README should contain the names of the teammates. (You don't have to use your real name if you don't want Microsoft/GitHub to know who you are.)
- Use the format `https://github.com/uofa-cmput404/f24-project-team-name` where `team-name` is your CSS colour.

Do not submit a clone link or a link to a branch or file.

- 1 mark
- Join your team on canvas
- Join your team's github via the github classroom link in the project part 0 assignment in canvas
- 1 GitHub repo with a README and LICENSE
- Team name is a CSS colour name

Project Part 1: Halfway Prototype

Requirements

For this part you need:

- ½ of the user stories except those marked for later parts, like "⧟ *Part 3-5 only.*"
    - They work in the UI.
    - They work in the local API (imagine you are building an API for a future Android client)
    - It's obvious how to perform the user stories in the UI.

    - They are tested. See the test requirements.
    - You can walk the TA through how do the user stories in the UI, in the API, and walk the TA through the relevant code, tests, and documentation.
        - Without bugs, snags, hiccups, or blockers!
- Everything implemented is also documented. See the documentation requirements.
    - There are examples in the API documentation for the ½ of the user stories chosen above.
- Code follows the code requirements.
- The UI is good-looking, easy to read, easy to navigate, easy to understand.
- Teammates are using Git and GitHub often and effectively according to the tool use requirements

Submission

Due 4PM.

Submit only the link to the commit in the following format:

https://github.com/uofa-cmput404/f24-project-orange/commit/8e2cac3a0c0fdd01f039ebe5269e1ee5952d9cad

- Every team member must submit the link to Canvas.
- If you don't submit the link to Canvas you will get a zero for the project part regardless of what your team gets.
- Submitted frontend framework form if using a frontend framework. https://docs.google.com/forms/d/e/1FAIpQLSdB01QRXRJxnBJeFzYs9zAV7bj8IyWWGK7OanEhUyxpk9oWtw/viewform?usp=publish-editor

Marking

- *User stories that have an API must be tested through the API. Other user stories can be tested by directly calling Django code.*
- *Tests do not count unless they are accurately testing the functionality.*
- *Failure to commit and tag on time, failure to submit a link to the tag, etc. will result in an overall mark of zero for this project part: not just a zero for Tool Use.*
- *Force push, rebase, or other operations that destroy git history, along with forging git history, authorship, messages, dates, etc. will also result in an overall mark of zero for this project part, and you will be reported to the Faculty under the Student Academic Integrity Policy.*
- *1/2 of the user stories does not include those marked for ⊶ later project parts.*

- Excellent
    - User Stories UI: At least ½ of the user stories are usable using the UI.
    - User Stories API: At least ½ of the user stories with a relevant API are usable using the API. Adheres to the specification.
    - Test Cases: At least ½ of the user stories have tests written and pass those tests.
    - UI Design: Looks impressive! It's obvious how to do any of the ½ of the user stories.
    - Tool use: Use of Git, GitHub issues, etc. is Evidence and Obvious. Commits/merges/PRs are small and frequent. Commit messages are helpful information for teammates. All the teammates are working at the same time, and using GitHub **and git** to help communicate and improve teamwork. You laugh in the face of merge conflicts! Well organized repository and branches.
    - TA Walkthrough: Able to walk through ½ user stories with UI and API. No snags, bugs, last second workarounds, or missing pieces. Code is easily located. Documentation is easily located.
    - Web Service API Documentation: Well documented. Highly detailed. Clear descriptions, has useful example requests and responses from your API for every use of the API and ½ of user stories. If using automated documentation generation, there is extra documentation added on top of that.
    - Standards & Code Style: Adheres to standards, code is well organized and clean. Code is easy to read. Comments add to code readability when necessary. Code meets Python, JS, HTML and CS style guides. Excellent indentation, naming. Code units only do one thing. It's easy to find the code responsible for handling any of the ½ user stories.
- Good
    - User Stories UI: Almost ½ of the user stories are usable using the UI.
    - User Stories API: Almost ½ of the user stories with a relevant API are usable using the API. Adheres to the specification.
    - Test Cases: At least ½ of the user stories have tests written, most pass tests.
    - UI Design: Looks good. It's not obvious how you would use the UI to do some of ½ of the user stories.
    - Tool use: Frequent but inconsistent use of Git, etc. Commits/merges/PRs are medium and with inconsistent frequency. All the teammates are working at the same time. Using GitHub to help communicate. Well organized repository and branches.
    - TA Walkthrough: A couple of snags, bugs, last second workarounds.
    - Web Service API Documentation: Well documented, medium detail. A few things are unclear. Has example requests and responses from your API for every use of

the API and ½ of user stories. If using automated documentation generation, there is extra documentation added on top of that.
    - Standards & Code Style: Adheres to standards, code is well organized and clean. Missing comments. Some minor issues with code style. Code units only do one thing. It takes a little digging sometimes to find the code responsible for a user story.
- Satisfactory
    - User Stories UI: At least 3/8ths of the user stories are usable using the UI.
    - User Stories API: At least 3/8ths of the user stories with a relevant API are usable using the API. Very close to adhering to the specification.
    - Unsatisfactory: At least ¼ of the user with a relevant API stories are usable using the API. Some parts deviate from the specification.
    - Test Cases: At least 3/8ths of the user have tests written, most pass tests.
    - UI Design: HTML & CSS exists, it does not look good. Mostly easy to understand. It has issues. There is some way to do ½ of the user stories, even if it's not obvious.
    - Tool use: Uses Git, etc. Has some oversized merges. Commits/merges/PRs are large and far apart. All the teammates are working at the same time. Well organized repository and branches.
    - TA Walkthrough: Some snags bugs, last second workarounds, or tiny missing pieces.
    - Web Service API Documentation: Well documented, but missing details. Every endpoint is documented. Some things are unclear. Example requests and responses have a few issues. Slightly less than ½ of user stories have examples. If using automated documentation generation, there is extra documentation added on top of that.
    - Standards & Code Style: Adheres to standards, there is an attempt at organization. Commented out code, unreachable code, or code with no clear purpose. Code units only do one thing. Occasional spots of poor style. It's not clear where the code for a user story will be.
- Unsatisfactory
    - User Stories UI: At least ¼ of the user stories are usable using the UI.
    - User Stories API: At least 1/4ths of the user stories with a relevant API are usable using the API. Very close to adhering to the specification.
    - Test Cases: At least ¼ of the user have tests written, most pass tests.
    - UI Design: HTML & CSS exists. Major UI issues. Hard to navigate. Hard to understand. There is some way to do almost all the ½ of the user stories.
    - Tool use: Limited of tool use. Only one teammate seems to be working at a time. Git is disorganized.

    - TA Walkthrough: Many snags, bugs, last second workarounds, small missing pieces.
    - Web Service API Documentation: Low on details. Many things are unclear. Every endpoint is documented. Missing some example requests and responses. If using automated documentation generation, there is extra documentation added on top of that.
    - Standards & Code Style: Browser, Python, transpiler, bundler, etc. warnings. Code is poorly organized God files/classes/functions. Have to search or grep code to find the code responsible for something.
- Attempt
    - User Stories UI: Less than ¼ of the user stories are usable using the UI or major deviations from the user stories.
    - User Stories API: Less than ¼ of the user stories are usable using the API or major deviations from the specification.
    - Test Cases: Less than ¼ of the user stories are usable using the API or major deviations from the specification, or test cases are present but do not actually test what they say they are testing.
    - UI Design: Some pieces of the HTML & CSS exist, but some pieces of the HTML & CSS are missing that would be needed to do ½ of the user stories.
    - Tool use: Very limited of tool use. Git is disorganized. Git exists but it has built artifacts in it. Git has commits that do not contribute anything, but have large diffs, such as changing whitespace of every line in a file.
    - TA Walkthrough: Showstopper bugs. Major missing pieces.
    - Web Service API Documentation: Major pieces of documentation are missing or only has autogenerated documentation.
    - Standards & Code Style: HTML errors, browser errors, CSS errors, JS errors, Python errors. Code is disorganized. Code is hard to find. Code is missing.
- No attempt
    - User Stories UI: No user stories are usable using the UI.
    - User Stories API: No user stories are usable using the API.
    - Test Cases: No test cases are present.
    - UI Design: No HTML/CSS exists.
    - Tool use: Used file sharing, email attachments, sending files/code through discord, chat, messengers, or similar tools to work together. Git exists but it not being used, or it is being misused.
    - TA Walkthrough: No walkthrough. Unable to demo.
    - Web Service API Documentation: Documentation is missing or unable to find documentation.
    - Standards & Code Style: Project doesn't run/load.

Project Part 2: Centralized Prototype

Submission

Due 4PM.

Submit only the link to the commit in the following format:

https://github.com/uofa-cmput404/f24-project-orange/commit/8e2cac3a0c0fdd01f039ebe5269e1ee5952d9cad

- Every team member must submit the link to Canvas.

Marking

- *"All user stories" does not include those marked for ⊶ later project parts.*
- *Almost all ~ 95% of the user stories not marked for ⊶ later project parts.*
- Excellent
    - **Addressing Feedback**: TAs suggestions were implemented, TA approves of implementation set.
    - User Stories UI: All the user stories are usable using the UI.
    - User Stories API: All the user stories with a relevant API are usable using the API. Adheres to the specification.
    - Test Cases: All the of the user stories have tests written and pass those tests.
    - UI Design: Looks impressive! It's obvious how to do any of the user stories.
    - Tool use: Use of Git, GitHub issues, etc. is Evidence and Obvious. Commits/merges/PRs are small and frequent. Commit messages are helpful information for teammates. All the teammates are working at the same time, and using GitHub **and git** to help communicate and improve teamwork. You laugh in the face of merge conflicts! Well organized repository and branches.
    - TA Walkthrough: Able to walk through every user story with UI and API. No snags, bugs, last second workarounds, or missing pieces. Code is easily located. Documentation is easily located.
    - Web Service API Documentation: Well documented. Highly detailed. Clear descriptions, has useful example requests and responses from your API for every use of the API for every user story. If using automated documentation generation, there is extra documentation added on top of that.
    - Standards & Code Style: Adheres to standards, code is well organized and clean. Code is easy to read. Comments add to code readablility when necessary. Code meets Python, JS, HTML and CS style guides. Excellent indentation, naming.

Code units only do one thing. It's easy to find the code responsible for handling any of the user stories.

- Good
    - **Addressing Feedback**: Almost all TA's suggestions were implemented, TA approves of implementation set.
    - User Stories UI: Almost all user stories are usable using the UI.
    - User Stories API: Almost all user stories with a relevant API are usable using the API. Adheres to the specification.
    - Test Cases: Almost all user stories have tests written, most pass tests.
    - UI Design: Looks good. It's not obvious how you would use the UI to do some of the user stories.
    - Tool use: Frequent but inconsistent use of Git, etc. Commits/merges/PRs are medium and with inconsistent frequency. All the teammates are working at the same time. Using GitHub to help communicate. Well organized repository and branches.
    - TA Walkthrough: A couple of snags, bugs, last second workarounds.
    - Web Service API Documentation: Well documented, medium detail. A few things are unclear. Has example requests and responses from your API for every use of the API and almost all the user stories. If using automated documentation generation, there is extra documentation added on top of that.
    - Standards & Code Style: Adheres to standards, code is well organized and clean. Missing comments. Some minor issues with code style. Code units only do one thing. It takes a little digging sometimes to find the code responsible for a user story.
- Satisfactory
    - **Addressing Feedback**: 3/4ths of TA's suggestions were implemented, TA approves of most of the changes made to address feedback.
    - User Stories UI: At least 3/4ths of the user stories are usable using the UI.
    - User Stories API: At least 3/4ths of the user stories with a relevant API are usable using the API. Very close to adhering to the specification.
    - Unsatisfactory: At least 3/4ths of the API with relevant user stories are usable using the API. Some parts deviate from the specification.
    - Test Cases: At least 3/4ths of the user have tests written, most pass tests.
    - UI Design: HTML & CSS exists, it does not look good. Mostly easy to understand. It has issues. There is some way to do all user stories, even if it's not obvious.
    - Tool use: Uses Git, etc. Has some oversized merges. Commits/merges/PRs are large and far apart. All the teammates are working at the same time. Well organized repository and branches.

    - TA Walkthrough: Some snags bugs, last second workarounds, or tiny missing pieces.
    - Web Service API Documentation: Well documented, but missing details. Every endpoint is documented. Some things are unclear. Example requests and responses have a few issues. Almost all user stories have examples. If using automated documentation generation, there is extra documentation added on top of that.
    - Standards & Code Style: Adheres to standards, there is an attempt at organization. Commented out code, unreachable code, or code with no clear purpose. Code units only do one thing. Occasional spots of poor style. It's not clear where the code for a user story will be.
- Unsatisfactory
    - **Addressing Feedback**: Most of TA's suggestions were implemented, TA approves of most of the changes made to address feedback.
    - User Stories UI: At least half of the user stories are usable using the UI.
    - User Stories API: At least half of the user stories with a relevant API are usable using the API. Very close to adhering to the specification.
    - Test Cases: At least half of the user have tests written, most pass tests.
    - UI Design: HTML & CSS exists. Major UI issues. Hard to navigate. Hard to understand. There is some way to do almost all user stories.
    - Tool use: Limited of tool use. Only one teammate seems to be working at a time. Git is disorganized.
    - TA Walkthrough: Many snags, bugs, last second workarounds, small missing pieces.
    - Web Service API Documentation: Low on details. Many things are unclear. Every endpoint is documented. Missing some example requests and responses. If using automated documentation generation, there is extra documentation added on top of that.
    - Standards & Code Style: Browser, Python, transpiler, bundler, etc. warnings. Code is poorly organized God files/classes/functions. Have to search or grep code to find the code responsible for something.
- Attempt
    - **Addressing Feedback**: Some of TA's suggestions were implemented, TA approves of some of the changes made to address feedback.
    - User Stories UI: Less than half of the user stories are usable using the UI or major deviations from the user stories.
    - User Stories API: Less than half of the user stories are usable using the API or major deviations from the specification.

    - Test Cases: Less than half of the user stories are usable using the API or major deviations from the specification, or test cases are present but do not actually test what they say they are testing.
    - UI Design: Some pieces of the HTML & CSS exist, but some pieces of the HTML & CSS are missing that would be needed to do some of the user stories.
    - Tool use: Very limited of tool use. Git is disorganized. Git exists but it has built artifacts in it. Git has commits that do not contribute anything, but have large diffs, such as changing whitespace of every line in a file.
    - TA Walkthrough: Showstopper bugs. Major missing pieces.
    - Web Service API Documentation: Major pieces of documentation are missing or only has autogenerated documentation.
    - Standards & Code Style: HTML errors, browser errors, CSS errors, JS errors, Python errors. Code is disorganized. Code is hard to find. Code is missing.
- No attempt
    - **Addressing Feedback**: None of TA's suggestions were implemented, TA approves of none of the changes made to address feedback.
    - User Stories UI: No user stories are usable using the UI.
    - User Stories API: No user stories are usable using the API.
    - Test Cases: No test cases are present.
    - UI Design: No HTML/CSS exists.
    - Tool use: Used file sharing, email attachments, sending files/code through discord, chat, messengers, or similar tools to work together. Git exists but it not being used, or it is being misused.
    - TA Walkthrough: No walkthrough. Unable to demo.
    - Web Service API Documentation: Documentation is missing or unable to find documentation.
    - Standards & Code Style: Project doesn't run/load.

Project Part 3: Distribution

Project part 3 includes user stories from part 1, user stories from part 2, and user stories marked with ⊶ new user stories. All of them. They should all be working with multiple nodes. All of them!

If a user story could work with multiple nodes, it should work with multiple nodes! For example:

- As an author, I want a "stream" which shows all the entries I should know about, so I don't have to switch between different pages.

This user story now also includes entries from remote nodes. The same principle applies to every user story in the list of user stories.

Requirements

- Everyone on the team must be able to deploy their own *node* (using the same project code) to their own Heroku "app" with their own Heroku Postgres database.
    - 1 codebase, 1 repo deployed to 6 nodes each on 6 different Heroku servers using 6 different databases, 6 different web servers, at 6 different addresses for 6 different team members.
    - **Please make sure each team member has deployed their node instance prior to your lab demo.**
- All nodes must be able coordinating. All user stories involving multiple authors must work in both situations:
    - All authors on the same node
    - Authors on different nodes
- For example if the user story is "As an author, I want to be able to approve or deny other authors following me, so that I don't get followed by people I don't like."
    - This needs to work when the author following is on a different node from the author they are following.
    - It also needs to work if they're on the same node.

Submission

Due 4PM.

Submit only the link to the commit in the following format:

https://github.com/uofa-cmput404/f24-project-orange/commit/8e2cac3a0c0fdd01f039ebe5269e1ee5952d9cad

- Every team member must submit the link to Canvas.

Marking

- **Everything / all user stories / almost all user stories, 3/4 user stories, etc. includes the ⊶ new user stories for project part 3 & part 4!**
- Excellent
    - **Web Service Coordination:** Each team member's node coordinates with the other team member's nodes successfully. All interoperation requirements met. No snags. All user stories that could involve two nodes work node-to-node.
    - Addressing Feedback: TA's suggestions were implemented, TA approves of implementation set.

    - User Stories UI: All the user stories are usable using the UI.
    - User Stories API: All the user stories with a relevant API are usable using the API. Adheres to the specification.
    - Test Cases: All the of the user stories have tests written and pass those tests.
    - UI Design: Looks impressive! It's obvious how to do any of the user stories.
    - Tool use: Use of Git, GitHub issues, etc. is Evidence and Obvious. Commits/merges/PRs are small and frequent. Commit messages are helpful information for teammates. All the teammates are working at the same time, and using GitHub **and git** to help communicate and improve teamwork. You laugh in the face of merge conflicts! Well organized repository and branches.
    - TA Walkthrough: Able to walk through every user story with UI and API. No snags, bugs, last second workarounds, or missing pieces. Code is easily located. Documentation is easily located.
    - Web Service API Documentation: Well documented. Highly detailed. Clear descriptions, has useful example requests and responses from your API for every use of the API for every user story. If using automated documentation generation, there is extra documentation added on top of that.
    - Standards & Code Style: Adheres to standards, code is well organized and clean. Code is easy to read. Comments add to code readablility when necessary. Code meets Python, JS, HTML and CS style guides. Excellent indentation, naming. Code units only do one thing. It's easy to find the code responsible for handling any of the user stories.
- Good
    - **Web Service Coordination:** Each team member's node coordinates with the other team member's nodes successfully. Almost all interoperation requirements met. Only some minor snags. Almost all user stories that could involve two nodes work node-to-node.
    - Addressing Feedback: Almost all TA's suggestions were implemented, TA approves of implementation set.
    - User Stories UI: Almost all user stories are usable using the UI.
    - User Stories API: Almost all user stories with a relevant API are usable using the API. Adheres to the specification.
    - Test Cases: Almost all user stories have tests written, most pass tests.
    - UI Design: Looks good. It's not obvious how you would use the UI to do some of the user stories.
    - Tool use: Frequent but inconsistent use of Git, etc. Commits/merges/PRs are medium and with inconsistent frequency. All the teammates are working at the same time. Using GitHub to help communicate. Well organized repository and branches.

    - TA Walkthrough: A couple of snags, bugs, last second workarounds.
    - Web Service API Documentation: Well documented, medium detail. A few things are unclear. Has example requests and responses from your API for every use of the API and almost all the user stories. If using automated documentation generation, there is extra documentation added on top of that.
    - Standards & Code Style: Adheres to standards, code is well organized and clean. Missing comments. Some minor issues with code style. Code units only do one thing. It takes a little digging sometimes to find the code responsible for a user story.
- Satisfactory
    - **Web Service Coordination:** Each team member's node coordinates with the other team member's nodes successfully. Most interoperation requirements met. Few snags. 3/4ths of user stories that could involve two nodes work node-to-node.
    - Addressing Feedback: 3/4ths of TA's suggestions were implemented, TA approves of most of the changes made to address feedback.
    - User Stories UI: At least 3/4ths of the user stories are usable using the UI.
    - User Stories API: At least 3/4ths of the user stories with a relevant API are usable using the API. Very close to adhering to the specification.
    - Unsatisfactory: At least 3/4ths of the API with relevant user stories are usable using the API. Some parts deviate from the specification.
    - Test Cases: At least 3/4ths of the user have tests written, most pass tests.
    - UI Design: HTML & CSS exists, it does not look good. Mostly easy to understand. It has issues. There is some way to do all user stories, even if it's not obvious.
    - Tool use: Uses Git, etc. Has some oversized merges. Commits/merges/PRs are large and far apart. All the teammates are working at the same time. Well organized repository and branches.
    - TA Walkthrough: Some snags bugs, last second workarounds, or tiny missing pieces.
    - Web Service API Documentation: Well documented, but missing details. Every endpoint is documented. Some things are unclear. Example requests and responses have a few issues. Almost all user stories have examples. If using automated documentation generation, there is extra documentation added on top of that.
    - Standards & Code Style: Adheres to standards, there is an attempt at organization. Commented out code, unreachable code, or code with no clear purpose. Code units only do one thing. Occasional spots of poor style. It's not clear where the code for a user story will be.

- Unsatisfactory
    - **Web Service Coordination:** The basics of coordination with another node are covered. Probably many snags. Most user stories that could involve two nodes work node-to-node.
    - Addressing Feedback: Most of TAs suggestions were implemented, TA approves of most of the changes made to address feedback.
    - User Stories UI: At least half of the user stories are usable using the UI.
    - User Stories API: At least half of the user stories with a relevant API are usable using the API. Very close to adhering to the specification.
    - Test Cases: At least half of the user have tests written, most pass tests.
    - UI Design: HTML & CSS exists. Major UI issues. Hard to navigate. Hard to understand. There is some way to do almost all user stories.
    - Tool use: Limited of tool use. Only one teammate seems to be working at a time. Git is disorganized.
    - TA Walkthrough: Many snags, bugs, last second workarounds, small missing pieces.
    - Web Service API Documentation: Low on details. Many things are unclear. Every endpoint is documented. Missing some example requests and responses. If using automated documentation generation, there is extra documentation added on top of that.
    - Standards & Code Style: Browser, Python, transpiler, bundler, etc. warnings. Code is poorly organized God files/classes/functions. Have to search or grep code to find the code responsible for something.
- Attempt
    - **Web Service Coordination:** Coordination with another node barely works. Less than half interoperation requirements met. Showstopper problems. Some user stories work node-to-node.
    - Addressing Feedback: Some of TA's suggestions were implemented, TA approves of some of the changes made to address feedback.
    - User Stories UI: Less than half of the user stories are usable using the UI or major deviations from the user stories.
    - User Stories API: Less than half of the user stories are usable using the API or major deviations from the specification.
    - Test Cases: Less than half of the user stories are usable using the API or major deviations from the specification, or test cases are present but do not actually test what they say they are testing.
    - UI Design: Some pieces of the HTML & CSS exist, but some pieces of the HTML & CSS are missing that would be needed to do some of the user stories.

    - Tool use: Very limited of tool use. Git is disorganized. Git exists but it has built artifacts in it. Git has commits that do not contribute anything, but have large diffs, such as changing whitespace of every line in a file.
    - TA Walkthrough: Showstopper bugs. Major missing pieces.
    - Web Service API Documentation: Major pieces of documentation are missing or only has autogenerated documentation.
    - Standards & Code Style: HTML errors, browser errors, CSS errors, JS errors, Python errors. Code is disorganized. Code is hard to find. Code is missing.
- No attempt
    - Web Service Coordination: No user stories are working between two nodes.
    - Addressing Feedback: None of TA's suggestions were implemented, TA approves of none of the changes made to address feedback.
    - User Stories UI: No user stories are usable using the UI.
    - User Stories API: No user stories are usable using the API.
    - Test Cases: No test cases are present.
    - UI Design: No HTML/CSS exists.
    - Tool use: Used file sharing, email attachments, sending files/code through discord, chat, messengers, or similar tools to work together. Git exists but it not being used, or it is being misused.
    - TA Walkthrough: No walkthrough. Unable to demo.
    - Web Service API Documentation: Documentation is missing or unable to find documentation.
    - Standards & Code Style: Project doesn't run/load.

Project Part 4: Federation

Requirements

NOTE: For part 4, select **one** node from part 3 to act as your team's main node. That is, your team should have one single node deployed on Heroku which meets the following requirements.

- Node is fully coordinating (all user stories) with nodes from **two** other teams.
- Node is partially (but still demonstrably some user stories) coordinating with nodes from **one** other team.
- Node is coordinating with least **three** other teams. 2+1=3
- All nodes must be able coordinating. All user stories involving multiple authors must work in both situations:
    - All authors on the same node
    - Authors on different nodes

- For example if the user story is "As an author, I want to be able to approve or deny other authors following me, so that I don't get followed by people I don't like."
    - This needs to work when the author following is on a different node from the author they are following.
    - It also needs to work if they're on the same node.

Submission

Due 4PM.

Submit only the link to the commit in the following format:

https://github.com/uofa-cmput404/f24-project-orange/commit/8e2cac3a0c0fdd01f039ebe5269e1ee5952d9cad

- Every team member must submit the link to Canvas.

Marking

- Everything / all user stories / almost all user stories, 3/4 user stories, etc. includes **all user stories that could involve two nodes, working with at least two other team's nodes!**
- Excellent
    - **Web Service Coordination:** Team's node **fully** coordinates with **two** other team's nodes successfully. All user stories that could involve two nodes work node-to-node with **two** other teams nodes.
    - **Inter-Team Coordination:** Demonstrable progress in getting node coordinating with **three** other teams nodes. (Includes the two teams above.)
    - Addressing Feedback: TA's suggestions were implemented, TA approves of implementation set.
    - User Stories UI: All the user stories are usable using the UI.
    - User Stories API: All the user stories with a relevant API are usable using the API. Adheres to the specification.
    - Test Cases: All the of the user stories have tests written and pass those tests.
    - UI Design: Looks impressive! It's obvious how to do any of the user stories.
    - Tool use: Use of Git, GitHub issues, etc. is Evidence and Obvious. Commits/merges/PRs are small and frequent. Commit messages are helpful information for teammates. All the teammates are working at the same time, and using GitHub **and git** to help communicate and improve teamwork. You laugh in the face of merge conflicts! Well organized repository and branches.

    - TA Walkthrough: Able to walk through every user story with UI and API. No snags, bugs, last second workarounds, or missing pieces. Code is easily located. Documentation is easily located.
    - Web Service API Documentation: Well documented. Highly detailed. Clear descriptions, has useful example requests and responses from your API for every use of the API for every user story. If using automated documentation generation, there is extra documentation added on top of that.
    - Standards & Code Style: Adheres to standards, code is well organized and clean. Code is easy to read. Comments add to code readablility when necessary. Code meets Python, JS, HTML and CS style guides. Excellent indentation, naming. Code units only do one thing. It's easy to find the code responsible for handling any of the user stories.
- Good
    - **Web Service Coordination:** Team's node coordinates with **two** other team's nodes successfully. Almost all interoperation requirements met. Only some minor snags. Almost all user stories that could involve two nodes work node-to-node between this team and two other team's nodes.
    - **Inter-Team Coordination:** Evidence of progress working with **three** other teams nodes. (Three includes the two teams above.)
    - Addressing Feedback: Almost all TA's suggestions were implemented, TA approves of implementation set.
    - User Stories UI: Almost all user stories are usable using the UI.
    - User Stories API: Almost all user stories with a relevant API are usable using the API. Adheres to the specification.
    - Test Cases: Almost all user stories have tests written, most pass tests.
    - UI Design: Looks good. It's not obvious how you would use the UI to do some of the user stories.
    - Tool use: Frequent but inconsistent use of Git, etc. Commits/merges/PRs are medium and with inconsistent frequency. All the teammates are working at the same time. Using GitHub to help communicate. Well organized repository and branches.
    - TA Walkthrough: A couple of snags, bugs, last second workarounds.
    - Web Service API Documentation: Well documented, medium detail. A few things are unclear. Has example requests and responses from your API for every use of the API and almost all the user stories. If using automated documentation generation, there is extra documentation added on top of that.
    - Standards & Code Style: Adheres to standards, code is well organized and clean. Missing comments. Some minor issues with code style. Code units only do one

thing. It takes a little digging sometimes to find the code responsible for a user story.

- Satisfactory
    - **Web Service Coordination:** Team's node coordinates with **one** other team's nodes successfully. Most interoperation requirements met. Few snags. 3/4ths of user stories that could involve two nodes work node-to-node with **one** other teams.
    - **Inter-Team Coordination:** Evidence of progress working with **three** other teams nodes. (Three includes the two teams above.)
    - Addressing Feedback: 3/4ths of TA's suggestions were implemented, TA approves of most of the changes made to address feedback.
    - User Stories UI: At least 3/4ths of the user stories are usable using the UI.
    - User Stories API: At least 3/4ths of the user stories with a relevant API are usable using the API. Very close to adhering to the specification.
    - Unsatisfactory: At least 3/4ths of the API with relevant user stories are usable using the API. Some parts deviate from the specification.
    - Test Cases: At least 3/4ths of the user have tests written, most pass tests.
    - UI Design: HTML & CSS exists, it does not look good. Mostly easy to understand. It has issues. There is some way to do all user stories, even if it's not obvious.
    - Tool use: Uses Git, etc. Has some oversized merges. Commits/merges/PRs are large and far apart. All the teammates are working at the same time. Well organized repository and branches.
    - TA Walkthrough: Some snags bugs, last second workarounds, or tiny missing pieces.
    - Web Service API Documentation: Well documented, but missing details. Every endpoint is documented. Some things are unclear. Example requests and responses have a few issues. Almost all user stories have examples. If using automated documentation generation, there is extra documentation added on top of that.
    - Standards & Code Style: Adheres to standards, there is an attempt at organization. Commented out code, unreachable code, or code with no clear purpose. Code units only do one thing. Occasional spots of poor style. It's not clear where the code for a user story will be.
- Unsatisfactory
    - **Web Service Coordination:** The basics of coordination with **one** other team's node is covered. Probably many snags. Most user stories that could involve two nodes work node-to-node with **one** other team.

    - **Inter-Team Coordination:** Evidence of progress working with **three** other teams nodes. (Three includes the one team above.)
    - Addressing Feedback: Most of TAs suggestions were implemented, TA approves of most of the changes made to address feedback.
    - User Stories UI: At least half of the user stories are usable using the UI.
    - User Stories API: At least half of the user stories with a relevant API are usable using the API. Very close to adhering to the specification.
    - Test Cases: At least half of the user have tests written, most pass tests.
    - UI Design: HTML & CSS exists. Major UI issues. Hard to navigate. Hard to understand. There is some way to do almost all user stories.
    - Tool use: Limited of tool use. Only one teammate seems to be working at a time. Git is disorganized.
    - TA Walkthrough: Many snags, bugs, last second workarounds, small missing pieces.
    - Web Service API Documentation: Low on details. Many things are unclear. Every endpoint is documented. Missing some example requests and responses. If using automated documentation generation, there is extra documentation added on top of that.
    - Standards & Code Style: Browser, Python, transpiler, bundler, etc. warnings. Code is poorly organized God files/classes/functions. Have to search or grep code to find the code responsible for something.
- Attempt
    - **Web Service Coordination:** Coordination with another team's node barely works. Less than half interoperation requirements met. Showstopper problems. Some user stories work node-to-node.
    - **Inter-Team Coordination:** Evidence of progress working with **two** other teams nodes. (Three includes the one team above.)
    - Addressing Feedback: Some of TA's suggestions were implemented, TA approves of some of the changes made to address feedback.
    - User Stories UI: Less than half of the user stories are usable using the UI or major deviations from the user stories.
    - User Stories API: Less than half of the user stories are usable using the API or major deviations from the specification.
    - Test Cases: Less than half of the user stories are usable using the API or major deviations from the specification, or test cases are present but do not actually test what they say they are testing.
    - UI Design: Some pieces of the HTML & CSS exist, but some pieces of the HTML & CSS are missing that would be needed to do some of the user stories.

    - Tool use: Very limited of tool use. Git is disorganized. Git exists but it has built artifacts in it. Git has commits that do not contribute anything, but have large diffs, such as changing whitespace of every line in a file.
    - TA Walkthrough: Showstopper bugs. Major missing pieces.
    - Web Service API Documentation: Major pieces of documentation are missing or only has autogenerated documentation.
    - Standards & Code Style: HTML errors, browser errors, CSS errors, JS errors, Python errors. Code is disorganized. Code is hard to find. Code is missing.
- No attempt
    - Web Service Coordination: No user stories are coordinating with another team's node.
    - Addressing Feedback: None of TA's suggestions were implemented, TA approves of none of the changes made to address feedback.
    - Inter-Team Coordination: No attempt to work with another team.
    - User Stories UI: No user stories are usable using the UI.
    - User Stories API: No user stories are usable using the API.
    - Test Cases: No test cases are present.
    - UI Design: No HTML/CSS exists.
    - Tool use: Used file sharing, email attachments, sending files/code through discord, chat, messengers, or similar tools to work together. Git exists but it not being used, or it is being misused.
    - TA Walkthrough: No walkthrough. Unable to demo.
    - Web Service API Documentation: Documentation is missing or unable to find documentation.
    - Standards & Code Style: Project doesn't run/load.

Project Part 5: Polish

Submission

Due 4PM.

Submit only the link to the commit in the following format:

https://github.com/uofa-cmput404/f24-project-orange/commit/8e2cac3a0c0fdd01f039ebe5269e1ee5952d9cad

- Every team member must submit the link to Canvas.

Presentation

Your presentation will be 5 minutes max. Then you will have a minute or two for questions.

Your presentation should focus on:

- Important functions
    - You won't have time for every function in 5 minutes, so make sure to cover the most important functions.
- Your UI
    - This is what makes your project different from the other teams, so be sure to show it off!
    - We don't really need to see the Django admin panel.
- Federation
    - Walk through your working connection with other team(s).

Guide for a good presentation:

- Ideally, everyone in your team should take turns speaking.
    - However, I do understand that some people have public speaking anxiety, so you won't be penalized if someone doesn't speak.
- Ideally, your presentation will use your live web app.
    - The most successful teams in the past use note cards.
    - Prepare backup slides in case something goes wrong that prevents you from doing a live demo.
- The best presentations tell a story, walking through a typical user scenario.
    - An example of such a scenario is the one in the "Scenario" section above. But don't use that one, come up with your own that shows off your app.
- Rehearse your presentation in person as a team to make sure it is close to 5 minutes.
- Make sure your team has a laptop that works with the projector *before* the presentation day.

During the other team's presentations:

- Rudeness to the other teams will not be tolerated.
- Your team must arrive on time. Late teams will not be allowed to present.
- You and your team must stay for all presentations and show respect, support, and appreciation for the other teams hard work.
- You may ask thoughtful questions after each presentation.
- Wait until after questions to applaud. Otherwise, everyone ends up awkwardly applauding each team twice.

Marking

- Excellent
    - **Presentation:** Presentation within time, shows excellent teamwork, promotes the application, uses the live application to show functionality.
    - **Video**: Video is well presented and not boring, less than 2 minutes. It is fun promotes your project. Makes people want to try your app out. Posted to the discussion forum thread to share it with everyone in the class.
    - Web Service Coordination: Team's node coordinates with **two** other team's nodes successfully. All user stories that could involve two nodes work node-to-node with **two** other teams nodes.
    - Inter-Team Coordination: Significant progress in getting node coordinating with **three** other teams nodes. (Three includes the two teams above.)
    - Addressing Feedback: TA's suggestions were implemented, TA approves of implementation set.
    - User Stories UI: All the user stories are usable using the UI.
    - User Stories API: All the user stories with a relevant API are usable using the API. Adheres to the specification.
    - Test Cases: All the of the user stories have tests written and pass those tests.
    - UI Design: Looks impressive! It's obvious how to do any of the user stories.
    - Tool use: Use of Git, GitHub issues, etc. is Evidence and Obvious. Commits/merges/PRs are small and frequent. Commit messages are helpful information for teammates. All the teammates are working at the same time, and using GitHub **and git** to help communicate and improve teamwork. You laugh in the face of merge conflicts! Well organized repository and branches.
    - TA Walkthrough: Able to walk through every user story with UI and API. No snags, bugs, last second workarounds, or missing pieces. Code is easily located. Documentation is easily located.
    - Web Service API Documentation: Well documented. Highly detailed. Clear descriptions, has useful example requests and responses from your API for every use of the API for every user story. If using automated documentation generation, there is extra documentation added on top of that.
    - Standards & Code Style: Adheres to standards, code is well organized and clean. Code is easy to read. Comments add to code readablility when necessary. Code meets Python, JS, HTML and CS style guides. Excellent indentation, naming. Code units only do one thing. It's easy to find the code responsible for handling any of the user stories.
- Good
    - **Presentation:** Presentation within time, some teamwork, promotes the application, uses the live application to show functionality, a couple rough spots.

    - **Video**: Video is well presented and not boring, less than 2 minutes. Contains details author would not care about. Posted to the discussion forum thread to share it with everyone in the class.
    - Web Service Coordination: Team's node coordinates with **two** other team's nodes successfully. Almost all interoperation requirements met. Only some minor snags. Almost all user stories that could involve two nodes work node-to-node between this team and two other team's nodes.
    - Inter-Team Coordination: Evidence of progress working with **three** other teams nodes. (Includes the two teams above.)
    - Addressing Feedback: Almost all TA's suggestions were implemented, TA approves of implementation set.
    - User Stories UI: Almost all user stories are usable using the UI.
    - User Stories API: Almost all user stories with a relevant API are usable using the API. Adheres to the specification.
    - Test Cases: Almost all user stories have tests written, most pass tests.
    - UI Design: Looks good. It's not obvious how you would use the UI to do some of the user stories.
    - Tool use: Frequent but inconsistent use of Git, etc. Commits/merges/PRs are medium and with inconsistent frequency. All the teammates are working at the same time. Using GitHub to help communicate. Well organized repository and branches.
    - TA Walkthrough: A couple of snags, bugs, last second workarounds.
    - Web Service API Documentation: Well documented, medium detail. A few things are unclear. Has example requests and responses from your API for every use of the API and almost all the user stories. If using automated documentation generation, there is extra documentation added on top of that.
    - Standards & Code Style: Adheres to standards, code is well organized and clean. Missing comments. Some minor issues with code style. Code units only do one thing. It takes a little digging sometimes to find the code responsible for a user story.
- Satisfactory
    - **Presentation**: Presentation nearly within time, some teamwork, reasonable presentation. Uses the live application to show functionality, a few snags.
    - **Video**: Video less than 2 minutes. Video shows the functionality of the app.
    - Web Service Coordination: Team's node coordinates with **one** other team's nodes successfully. Most interoperation requirements met. Few snags. 3/4ths of user stories that could involve two nodes work node-to-node with **one** other teams.

    - Inter-Team Coordination: Evidence of progress working with **three** other teams nodes. (Three includes the two teams above.)
    - Addressing Feedback: 3/4ths of TA's suggestions were implemented, TA approves of most of the changes made to address feedback.
    - User Stories UI: At least 3/4ths of the user stories are usable using the UI.
    - User Stories API: At least 3/4ths of the user stories with a relevant API are usable using the API. Very close to adhering to the specification.
    - Unsatisfactory: At least 3/4ths of the API with relevant user stories are usable using the API. Some parts deviate from the specification.
    - Test Cases: At least 3/4ths of the user have tests written, most pass tests.
    - UI Design: HTML & CSS exists, it does not look good. Mostly easy to understand. It has issues. There is some way to do all user stories, even if it's not obvious.
    - Tool use: Uses Git, etc. Has some oversized merges. Commits/merges/PRs are large and far apart. All the teammates are working at the same time. Well organized repository and branches.
    - TA Walkthrough: Some snags bugs, last second workarounds, or tiny missing pieces.
    - Web Service API Documentation: Well documented, but missing details. Every endpoint is documented. Some things are unclear. Example requests and responses have a few issues. Almost all user stories have examples. If using automated documentation generation, there is extra documentation added on top of that.
    - Standards & Code Style: Adheres to standards, there is an attempt at organization. Commented out code, unreachable code, or code with no clear purpose. Code units only do one thing. Occasional spots of poor style. It's not clear where the code for a user story will be.
- Unsatisfactory
    - **Presentation:** A presentation was prepared but has problems. Low teamwork. Uses slides. Major snags.
    - **Video**: Video longer than 2 minutes. Video misrepresents the functionality of the app.
    - Web Service Coordination: The basics of coordination with **one** other team's node is covered. Probably many snags. Most user stories that could involve two nodes work node-to-node with **one** other team.
    - Inter-Team Coordination: Evidence of progress working with **three** other teams nodes. (Three includes the one team above.)
    - Addressing Feedback: Most of TAs suggestions were implemented, TA approves of most of the changes made to address feedback.

    - User Stories UI: At least half of the user stories are usable using the UI.
    - User Stories API: At least half of the user stories with a relevant API are usable using the API. Very close to adhering to the specification.
    - Test Cases: At least half of the user have tests written, most pass tests.
    - UI Design: HTML & CSS exists. Major UI issues. Hard to navigate. Hard to understand. There is some way to do almost all user stories.
    - Tool use: Limited of tool use. Only one teammate seems to be working at a time. Git is disorganized.
    - TA Walkthrough: Many snags, bugs, last second workarounds, small missing pieces.
    - Web Service API Documentation: Low on details. Many things are unclear. Every endpoint is documented. Missing some example requests and responses. If using automated documentation generation, there is extra documentation added on top of that.
    - Standards & Code Style: Browser, Python, transpiler, bundler, etc. warnings. Code is poorly organized God files/classes/functions. Have to search or grep code to find the code responsible for something.
- Attempt
    - **Presentation**: Lack of practice, lack of preparation, irrelevant, many snags, includes video.
    - **Video**: A video of the app exists.
    - Web Service Coordination: Coordination with another team's node barely works. Less than half interoperation requirements met. Showstopper problems. Some user stories work node-to-node.
    - Inter-Team Coordination: Evidence of progress working with **two** other teams nodes. (Three includes the one team above.)
    - Addressing Feedback: Some of TA's suggestions were implemented, TA approves of some of the changes made to address feedback.
    - User Stories UI: Less than half of the user stories are usable using the UI or major deviations from the user stories.
    - User Stories API: Less than half of the user stories are usable using the API or major deviations from the specification.
    - Test Cases: Less than half of the user stories are usable using the API or major deviations from the specification, or test cases are present but do not actually test what they say they are testing.
    - UI Design: Some pieces of the HTML & CSS exist, but some pieces of the HTML & CSS are missing that would be needed to do some of the user stories.

    - Tool use: Very limited of tool use. Git is disorganized. Git exists but it has built artifacts in it. Git has commits that do not contribute anything, but have large diffs, such as changing whitespace of every line in a file.
    - TA Walkthrough: Showstopper bugs. Major missing pieces.
    - Web Service API Documentation: Major pieces of documentation are missing or only has autogenerated documentation.
    - Standards & Code Style: HTML errors, browser errors, CSS errors, JS errors, Python errors. Code is disorganized. Code is hard to find. Code is missing.
- No attempt
    - **Presentation**: No presentation.
    - **Video**: No video of the app exists.
    - Web Service Coordination: No user stories are coordinating with another team's node.
    - Addressing Feedback: None of TA's suggestions were implemented, TA approves of none of the changes made to address feedback.
    - Inter-Team Coordination: No attempt to work with another team.
    - User Stories UI: No user stories are usable using the UI.
    - User Stories API: No user stories are usable using the API.
    - Test Cases: No test cases are present.
    - UI Design: No HTML/CSS exists.
    - Tool use: Used file sharing, email attachments, sending files/code through discord, chat, messengers, or similar tools to work together. Git exists but it not being used, or it is being misused.
    - TA Walkthrough: No walkthrough. Unable to demo.
    - Web Service API Documentation: Documentation is missing or unable to find documentation.
    - Standards & Code Style: Project doesn't run/load.